OpenAI Codex v C++ týmu

Agentní vývoj od zadání po merge request · 1. den

Ing. Jan Polišenský

9. 11. 2026 · privátní kurz pro Thermo Fisher Scientific



Ing. Jan Polišenský

CTO Lakmoos AI | AI Researcher | University Lecturer

O mně

- Výzkumník AI na VUT FIT v Brně
- Specializace: neurosymbolická AI, LLM, detekce hrozeb
- Aktivní přednášející na univerzitách (např. Oxford)
- Publikace, akademický výzkum, vedení diplomových prací

Lakmoos AI (CTO & co-founder)

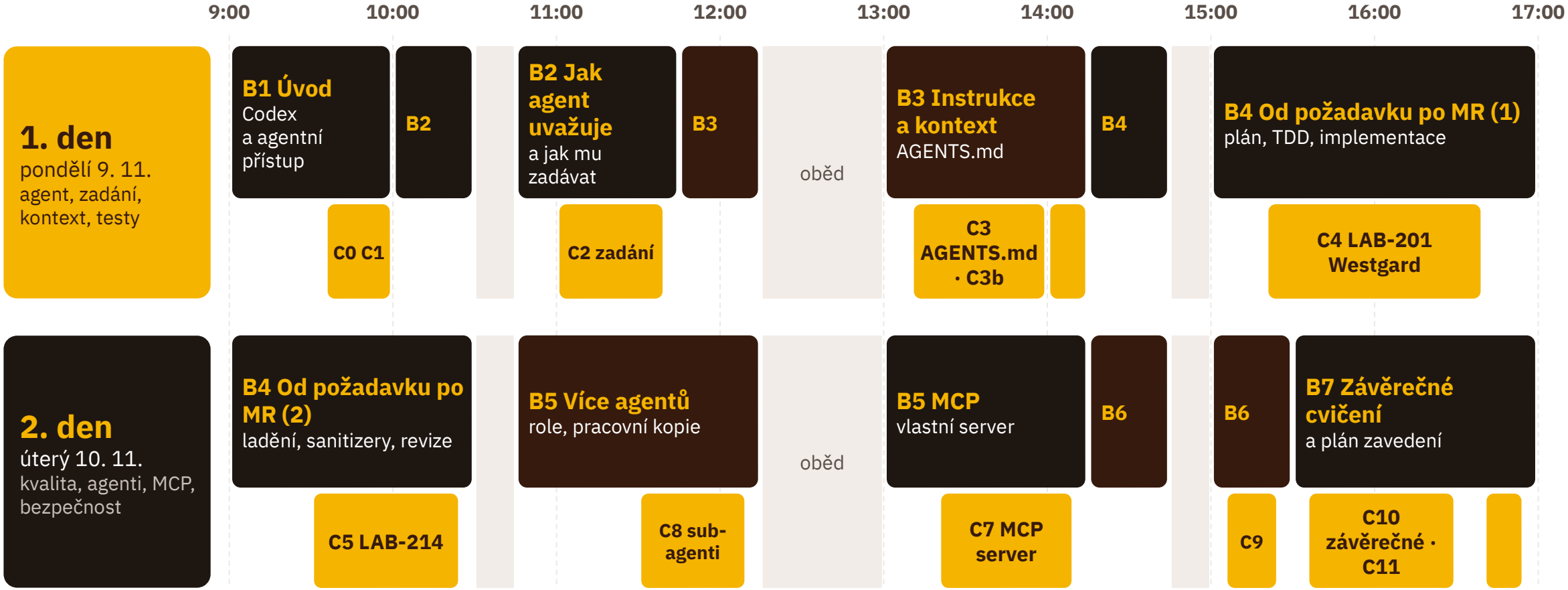
- Vedu tým ~30 inženýrů
- 3× Slush Top 50 inovativních startupů
- Finalista Nápad roku 2024
- Agenty pro programování používáme každý den ve vlastním vývoji



jan@lakmoos.com

Dva dny, jeden průběžný příklad

Žluté pruhy jsou cvičení. Každý blok jimi končí.



Tiket LAB-201 dnes projde od zadání po hotové testy. Zítra hledáme skutečnou chybu.

Jak budeme pracovat

Školení je pro vás, ne pro slidy

1	**Ptejte se kdykoliv**	Otázka uprostřed bloku je lepší než otázka, která zapadne.	*co nesouvisí, zapíšu na flipchart a vrátíme se k tomu*
2	**Prakticky**	Každý blok končí cvičením nad stejným repozitářem.	*dnes cvičení C0 až C4*
3	**Codex u vás, ukázky u lektora**	Ukázky běží v kontejneru u lektora, vy pracujete ve svém prostředí.	*lokální Docker nepotřebujete*
4	**Vzorový kód, ne firemní**	sample-service je smyšlená laboratorní služba v C++20.	*smíte ji dát jakémukoli agentovi*
5	**Nadšenci i skeptici**	Obojí je v pořádku. Cílem je společný postup týmu.	*ne přesvědčování*

Skepse je vítaná: tým, kde všichni jen jásají, přehlíží limity.

sample-service: o čem je vzorová služba

Fronta vzorků laboratorního analyzátoru, přístroj je simulovaný

01

SUBMIT

Vzorek do fronty s prioritou STAT, URGENT nebo ROUTINE. ID vypadá jako S-004711.

02

LOAD

Fronta plní volné pozice stojanu, STAT má přednost.

03

RUN

Přístroj pošle řádek firmwaru, kalibrační křivka převede signál na koncentraci.

04

Výsledek

JSON řádek s příznaky rozsahu (L, H, D), na požádání CSV report.

Deterministická doména: stejný vstup, stejný výstup. Ideální pro testy.

Postavit a otestovat jedním příkazem

Materiály, zadání cvičení a vzorový projekt jsou na GitHubu

```
# repozitář z GitHubu
git clone https://github.com/poli-cz/sample-service-codex-kurz sample-service
cd sample-service

# konfigurace, build a všechny testy
python scripts/check_all.py debug
  ...
  100% tests passed, 0 tests failed out of 49

# mezi pokusy zpět na začátek
git restore .
git clean -fd
```

- **C++20, CMake 3.22+**
  překladač s C++20 (např. GCC 12+, Clang 16+), Ninja
- **Knihovny i bez instalace**
  {fmt}, nlohmann_json, GoogleTest v third_party/
- **Šest presetů**
  debug, release, asan, tidy, coverage, vcpkg
- **49 testů**
  GoogleTest, Python skripty a smoke test služby

github.com/poli-cz/sample-service-codex-kurz

Kdo nemá build zelený do první přestávky, pracuje ve dvojici. Kontrola prostředí je cvičení C0.

Kolik kódu dnes píšete s AI?

Zvedněte ruku u jednoho písmene

A

Skoro nic

Vyzkoušeno, nesedlo to.

B

Občas

Na dotazy a kousky kódu.

C

Denně

Codex je běžná součást práce.

D

Agent sám

Zadám úlohu, kontroluji výsledek.

Žádná odpověď není špatně. Podle rozložení upravím tempo.

Dvě minuty se sousedem

1. Co vám s AI v C++ funguje?
2. Kde to drhne a proč?
3. Co po agentovi opravujete nejčastěji?



Tiket na celý den: LAB-201

Zkrácený překlad, plné znění v docs/tickets/LAB-201.md

LAB-201 | Otevřený | labcore | kvalita | nová funkce

Westgardova pravidla pro kontrolní vzorky

POPIS

Kontrolní vzorky mají známý cílový průměr a SD šarže, dnes je obsluha hodnotí okem. Přidejte do labcore funkci lab::qc::evaluate_westgard (hlavička labcore/qc_rules.hpp). Vstup: hodnoty v pořadí měření, průměr a SD. Výstup: seznam porušení {rule, index}. Pravidla 1-2s (varování), 1-3s, 2-2s, R-4s, 4-1s a 10-x.

AKCEPTAČNÍ KRITÉRIA

- [ ] „Za mezí“ = ostře větší; hodnota na průměru přeruší řadu 10-x
- [ ] Pravidlo se hlásí na každém indexu, 1-2s vždy, i vedle jiných
- [ ] SD <= 0 vyhodí std::invalid_argument, prázdný vstup = prázdný výsledek
- [ ] Testy napřed; debug a tidy projdou; žádné NOLINT ani -Wno-*

ČTĚTE JAKO AGENT

Rozhraní je přesné
Struktura Violation a podpis funkce jsou přímo v tiketu.

Hrany jsou rozhodnuté
Mez, průměr, pořadí výstupu. Proč na nich záleží: blok 2.

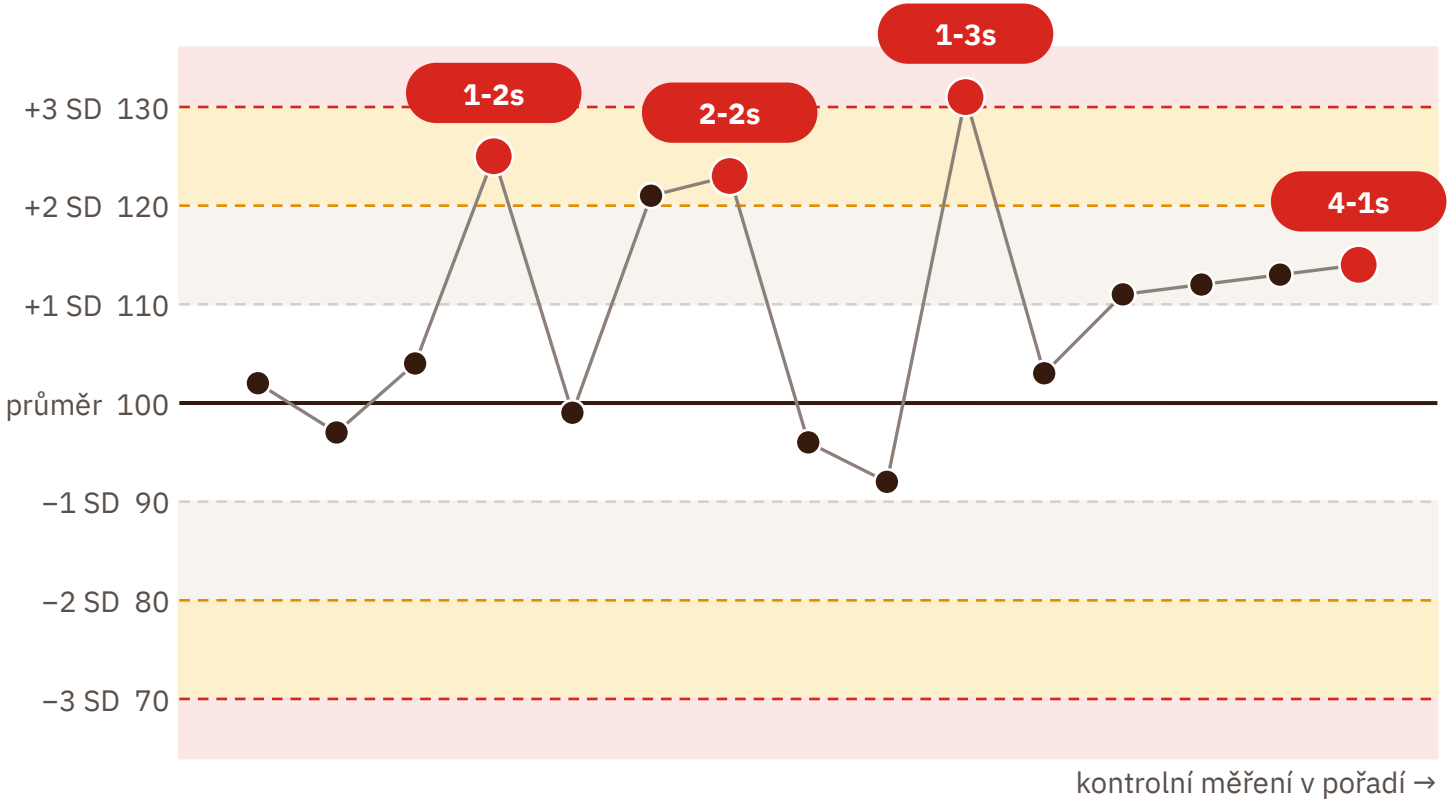
Co je doménové
Pravidla zná laborant. Agent je zná jen z tréninku.

Mimo rozsah
Napojení na službu a záznamy C|... přijde zítra.

Dobrý tiket je skoro hotové zadání. V bloku 2 uvidíte, co agent udělá bez něj.

Westgardova pravidla za minutu

Kontrolní vzorek se známou hodnotou: když se odchýlí, výsledky pacientů se nevydají



Zdroj: Westgard a kol., Clinical Chemistry 1981. Hodnoty v grafu jsou smyšlené.

PRAVIDLA V TIKETU

1-2s	jedna hodnota za ±2 SD: jen varování
1-3s	jedna hodnota za ±3 SD
2-2s	dvě po sobě za stejnou mezí 2 SD
R-4s	dvě po sobě: jedna za +2 SD, druhá za −2 SD
4-1s	čtyři po sobě za stejnou mezí 1 SD
10-x	deset po sobě na stejné straně průměru

Pravidla zná každý laborant. Agent je zná jen z tréninku, a to na testy nestačí.

Kudy LAB-201 dnes projde

Jedna úloha, pět pohledů



Agent čte repo

kde by LAB-201 žil

Zadání

slabé a dobré, cvičení C2

AGENTS.md

stejná úloha před a po

Testy a kód

TDD a implementace, C4

Rozšíření

příkaz QC nad záznamy C|..., závěrečné cvičení

Stejná čísla celý den: průměr 100, SD 10, tabulka očekávaných výsledků.

BLOK 1 ZE 7

Úvod: Codex a agentní přístup

01

Agent není doplňování kódu. Pracuje ve smyčce a vy ji řídíte.

60 MINUT · C0 A C1

Postup kurzu

Od doplňování kódu k agentům

Každý krok zpočátku vypadal jako hračka



GitHub Copilot

doplňuje řádky v editoru

ChatGPT

odpovídá v chatu, kód kopírujete

Codex CLI

agent v repozitáři, spouští build a testy

Více agentů

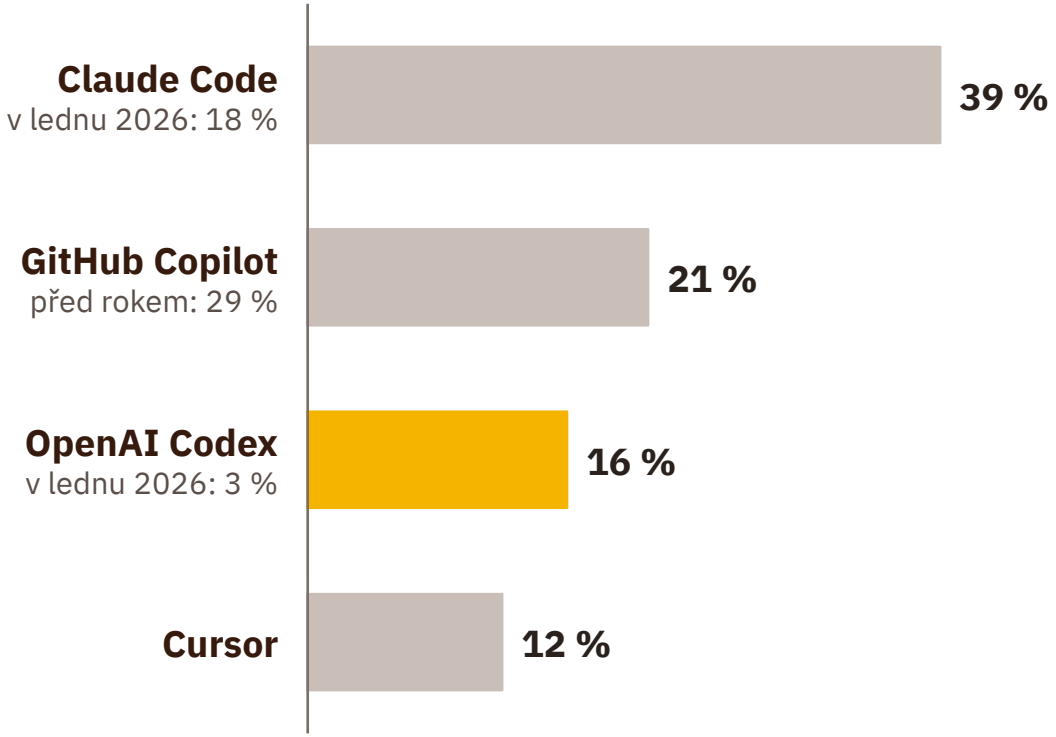
paralelní úlohy, vy řídíte a kontrolujete

Rozdíl není jen v modelu. Agent sám pracuje s repozitářem: čte, mění, spouští.

Agenti pro programování jsou běžná výbava

Které nástroje vývojáři používají v práci, květen až červenec 2026

PODÍL PROFESIONÁLNÍCH VÝVOJÁŘŮ, KTEŘÍ NÁSTROJ POUŽÍVAJÍ V PRÁCI



Průzkum JetBrains 2026

- Přes 15 000 profesionálních vývojářů
- 90 % používá agenty pro programování aspoň jednou týdně, 68 % denně
- Povědomí o Codexu: z 27 % na 65 %
- Lednová čísla jsou z jiného průzkumu (AI Pulse), srovnání je orientační

Zdroj: JetBrains Research: AI coding agent adoption 2026 (blog, 18. 8. 2026), Developer Ecosystem Survey 2026

Otázka už není „jestli agenty“, ale „jak s nimi pracovat jako tým“.

Nejste v tom sami

Průzkum mezi vývojáři, 2025

84 %

používá AI nástroje, nebo to plánuje

46 %

nedůvěřuje přesnosti výstupů

66 %

nejvíc vadí řešení „skoro správně, ale ne úplně“

Zdroj: Stack Overflow Developer Survey 2025

„Skoro správně“ je zkušenost většiny, ne vaše selhání.

Chat odpoví, agent pracuje

Agent = model + nástroje + smyčka + cíl

Chat

- Odpoví jednou na jeden dotaz
- Vidí jen to, co vložíte
- Nic nespustí ani neověří
- Kód kopírujete a testujete sami

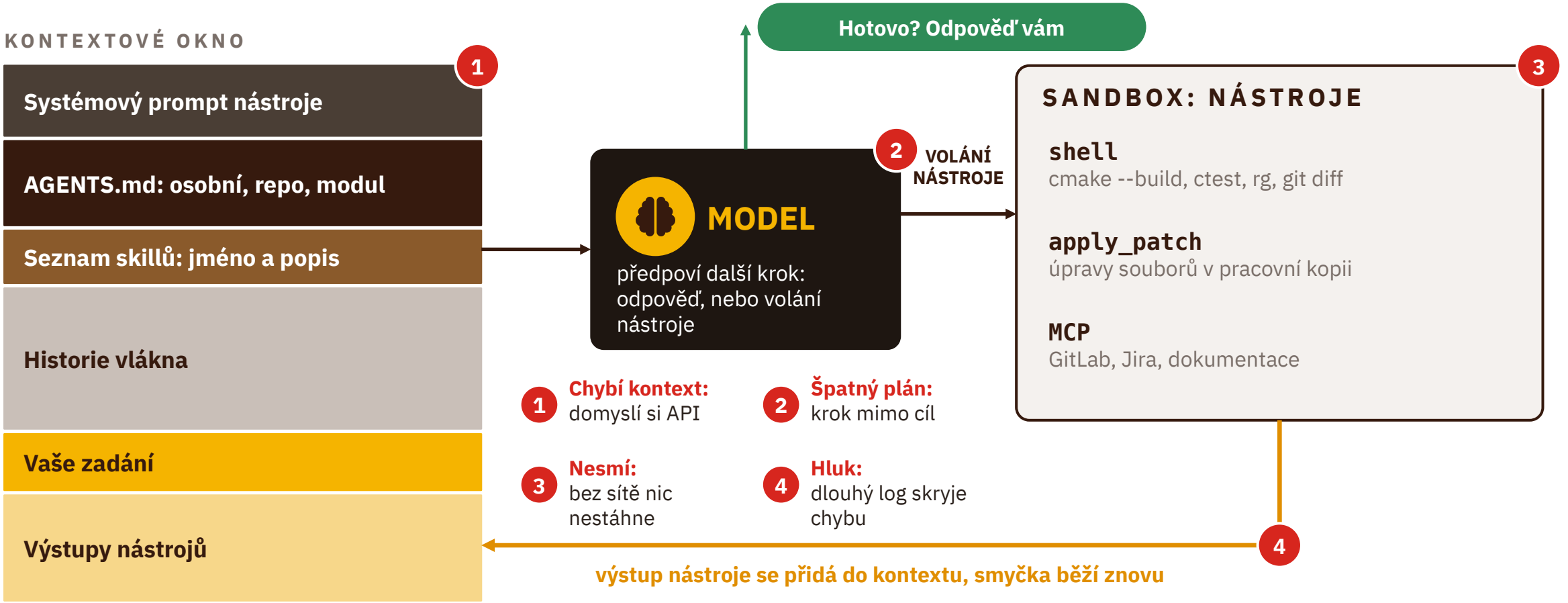
Agent

- Pracuje v krocích, dokud nemá pocit, že je hotovo
- Sám čte soubory v repozitáři
- Spouští build, testy a příkazy
- Mění soubory přímo ve vaší pracovní kopii

Model sám nic nespouští. Teprve nástroje z něj dělají agenta, proto záleží na oprávněních.

Jeden krok agenta zevnitř

Kontext, plán, akce přes nástroj, výstup zpět do kontextu



Každá chyba agenta má adresu: kontext, plán, nástroj, nebo výstup. Podle ní víte, co opravit.

Co agent skutečně dělá

Zkrácený průběh jedné úlohy v Codex CLI nad sample-service

```
› Přidej do summarize i směrodatnou odchylku
  kontrolní šarže

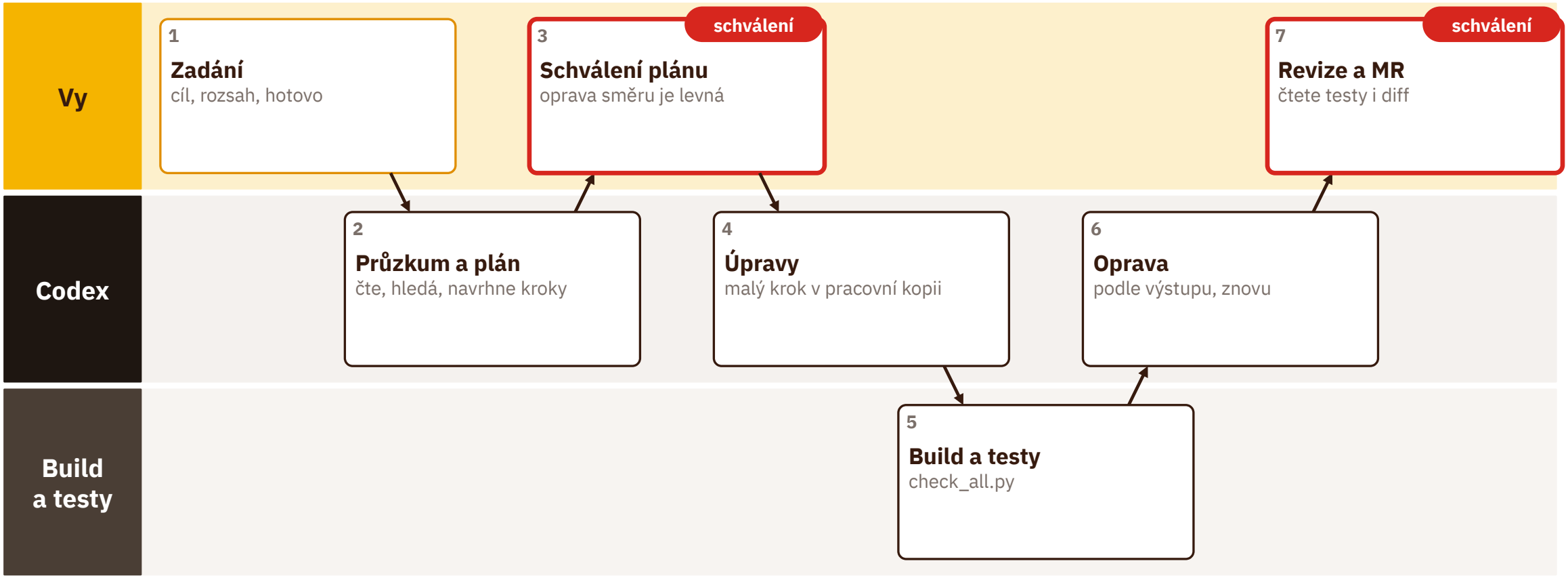
• Hledám: rg -n "stddev" src tests
• Čtu: src/labcore/include/labcore/statistics.hpp
• Upravuji: src/labcore/src/statistics.cpp (+9 −1)
• Spouštím: cmake --build --preset debug
  error: 'stddev' is not a member of 'lab::stats'
• Čtu: statistics.hpp, řádek 20: sample_stddev
• Upravuji: src/labcore/src/statistics.cpp (+1 −1)
• Spouštím: ctest --preset debug -R Statistics
  100% tests passed
✔ Hotovo: upraveno summarize, testy prošly
```

- **Hledání a čtení**
  kontext si skládá sám
- **Úprava souborů**
  přímo ve vaší pracovní kopii
- **Build a testy**
  chyba překladače je nová informace
- **Shrnutí**
  tvrzení, které je třeba ověřit

Každý řádek je volání nástroje. Výstup překladače je pro agenta nejcennější zpětná vazba.

Kde má být ve smyčce člověk

Dvě pevná místa: schválení plánu a revize diffu



Mezi dvěma schváleními může agent pracovat sám. Bez nich je to loterie.

Žebřík autonomie: kde je který nástroj

Každá příčka výš: víc práce bez vás a víc kontroly na konci

L1 Doplňování	L2 Řízená konverzace	L3 Samostatná úloha	L4 Více agentů
KDY	KDY	KDY	KDY
Píšete vy, nástroj dokončí řádek nebo funkci. Copilot, Cursor.	Chat nebo agent v editoru, každý krok schvalujete. ChatGPT, Copilot, Cursor.	Agent sám čte, mění soubory, spouští build a testy. Codex CLI, Claude Code.	Paralelní úlohy, sub-agenti, cloudová prostředí.
		dnes pracujeme tady	
`Tab`	`chat`	`codex`	`cloud`
člověk píše, nástroj nic neověří	člověk schvaluje každý krok	člověk schválí plán a čte diff	člověk řídí, kontroluje a slučuje

MÉNĚ SAMOSTATNOSTI → VÍC SAMOSTATNOSTI

Dnes pracujeme na L3, zítra na L4. Na další příčku vedou testy, ne odvaha.

Kde s Codexem pracujete?

Zvedněte ruku u jednoho písmene

A

Zatím nikde

Codex zatím nepoužívám.

B

Chat a editor

ChatGPT nebo Copilot, kód kopíruji.

C

VS Code

Rozšíření Codexu v editoru.

D

CLI nebo cloud

Codex v terminálu, úlohy v cloudu.

Dnes pracujeme hlavně v CLI. Postup je stejný i v rozšíření pro VS Code.

Čtyři prostředí, jeden agent

Stejný AGENTS.md a stejná pravidla, jiné místo běhu

	CLI	Rozšíření VS Code	Aplikace	Codex cloud
Kde běží kód	na vašem počítači	na vašem počítači	na vašem počítači	v kontejneru OpenAI
Síť pro agenta	v sandboxu vypnutá	v sandboxu vypnutá	v sandboxu vypnutá	během práce vypnutá
Repozitář	pracovní kopie	otevřený projekt	pracovní kopie	GitHub, GitLab (beta)
Hodí se na	ukázky a cvičení dnes	práce v editoru	přehled více vláken	paralelní úlohy

Dnes CLI. Cokoli se naučíte, platí i v rozšíření pro VS Code.

Spuštění Codex CLI: režimy, které platí

Ověřeno v Codex CLI 0.163.0

```
# jen čtení: orientace, plán, revize
codex -s read-only -a on-request

# úpravy v pracovní složce, na zbytek se ptá
codex -s workspace-write -a on-request

# schvalování převezme automatická revize
codex --approve-for-me

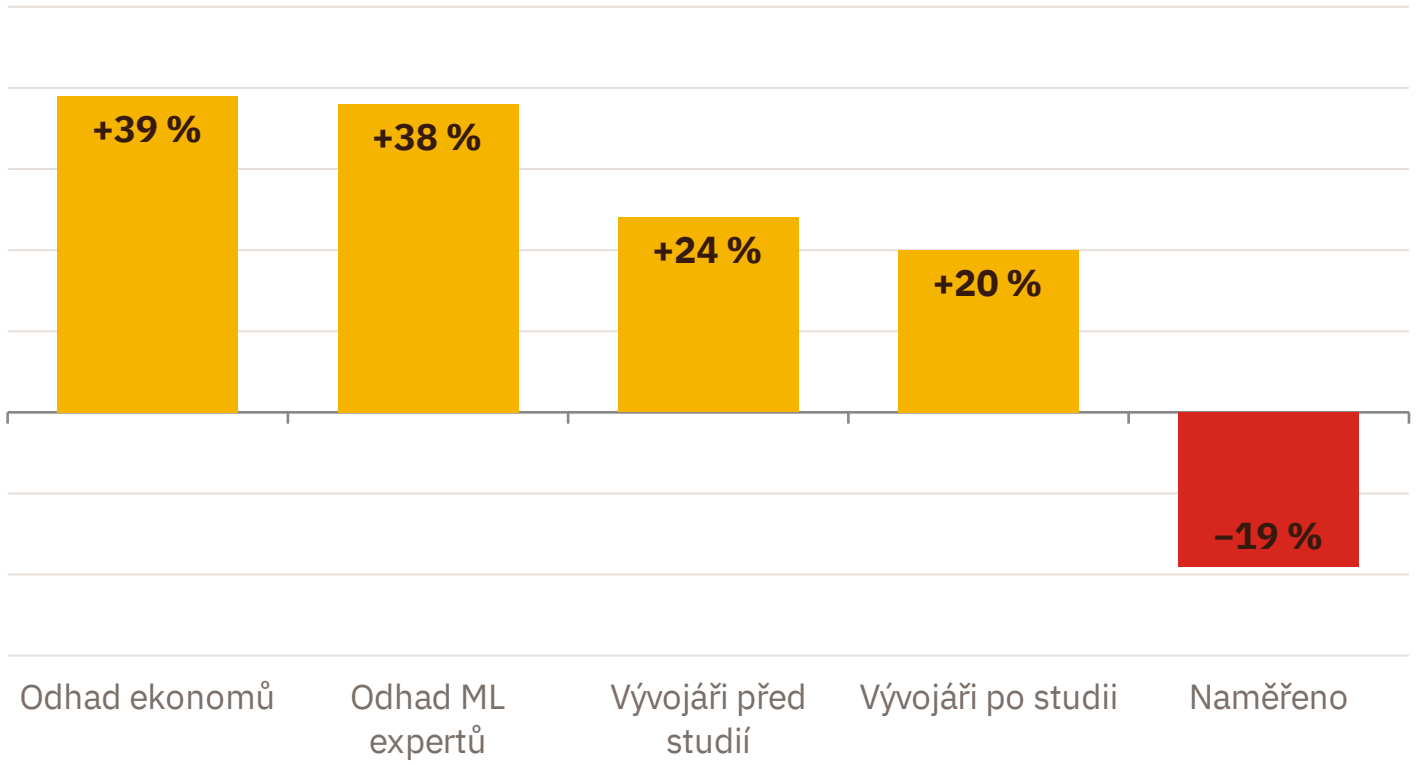
# staré návody, které už neplatí
codex --full-auto
error: unexpected argument '--full-auto' found
codex -c 'approval_policy="untrusted"'
Error: approval_policy = "untrusted" is no longer
  supported; remove this setting
```

- **-s, --sandbox**
  read-only, workspace-write, danger-full-access
- **-a, --ask-for-approval**
  jen on-request a never
- **--approve-for-me**
  workspace-write a automatická revize schvalování
- **Návody z internetu**
  často platí pro starší verze, ověřte je přes --help

Když návod nefunguje, nehádejte: codex --help a codex <příkaz> --help mají pravdu.

Pocit zrychlení není měření

Zkušení vývojáři na vlastních velkých repozitářích



Studie METR, 2025

- 16 zkušených open source vývojářů
- 246 skutečných úloh z jejich repozitářů
- Náhodně s AI a bez AI
- S AI byli o 19 % pomalejší, i když věřili, že jsou rychlejší

Zdroj: METR: Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity (2025)

Na velké známé kódové bázi AI zrychlení nezaručuje. Rozhoduje způsob práce, a ten dnes trénujeme.

Kde AI šetří čas a kde ho přidává

Zrychlení, riziko a cena ověření podle fáze vývoje

	ZRYCHLENÍ	RIZIKO CHYBY	CENA OVĚŘENÍ	CO S TÍM
Orientace v kódu	vysoké	nízké	nízké	Nejlepší start: agent jen čte a vysvětluje, nic nemění
Analýza požadavku	střední	střední	střední	Najde dotčená místa v kódu, rozhoduje člověk
Rutinní implementace	vysoké	střední	nízké	S testy a dobrým zadáním delegovat
Nová doménová logika	střední	vysoké	vysoké	Malé kroky, očekávané hodnoty určuje člověk
Testy	vysoké	střední	střední	Agent píše testy, hodnoty v nich kontroluje člověk
Paměťové chyby	střední	vysoké	střední	Bez sanitizeru agent chybu nevidí, s ním ano
Revize kódu	střední	nízké	nízké	Druhý pár očí před kolegou, ne místo něj
Dokumentace a MR	vysoké	nízké	střední	Vznikne rychle, ale někdo ji musí přečíst

Odhad lektora pro typický C++ tým. Zelená = výhodné, oranžová = záleží na postupu, červená = pozor.

Čas šetří tam, kde se výsledek levně ověří. Jinde se práce jen přesouvá do kontroly.

ictPRO

Co agent dnes zvládne a co ne

Stav k říjnu 2026, s dobrým zadáním a buildem, který smí spustit

Zvládne

Najít a přečíst kód
rg, hlavičky, CMakeLists rychleji než člověk

Psát podle vzoru
když vzor v repozitáři vidí

Opravit se podle chyby
výstup překladače a testů je nejlepší zpětná vazba

Navrhnout testy
scénáře a kostru, hodnoty kontrolujete vy

Sám nezvládne

Vědět, co nevidí
porady, nepsané konvence, produkční data

Rozhodnout hrany požadavku
jestli > nebo ≥ určuje laboratoř, ne model

Poznat paměťovou chybu bez sanitizeru
build i testy mohou projít

Nést odpovědnost
commit od agenta je váš commit

Role vývojáře se posouvá: zadávat, ověřovat, rozhodovat. Psaní ubývá, kontroly přibývá.

CVIČENÍ C0 a C1 · 25 MINUT

Prostředí a první úloha

Nejdřív zelený build, pak agent, který jen čte

CÍL

Ověřit, že Codex, překladač a build fungují, a pak nechat Codex popsat architekturu sample-service bez jediné změny v repozitáři.

`cviceni/00-priprava, cviceni/01-prvni-uloha`

POSTUP

1. C0: skript, build, asan a tidy (10 min)
2. C0: jeden dotaz Codexu jen pro čtení
3. C1: Codex jen pro čtení, vložit zadání (15 min)
4. C1: sledovat, co agent čte a spouští
5. C1: porovnat s README a docs/architecture.md

ODEVZDÁTE

Zelený build se 49 testy a odpovědi na pět otázek C1 s odkazy na soubory a funkce.

HOTOVO, KDYŽ

skript hlásí 0 chyb, debug, asan a tidy projdou, Codex odpoví a git status je čistý.

ictPRO

Kontrola prostředí

Codex, Git, CMake, překladač a zelený build

1 C0 Kontrola prostředí | 2 C1 První úloha jen pro čtení

CO UDĚLAT

1. Skript over_prostredi.py jen kontroluje, nic neinstaluje.
2. Klon z GitHubu a build jedním příkazem, pak asan a tidy.
3. Jedno zadání v Codexu ověří licenci i síť.
4. Prostředí pro MCP server (krok 4 v ZADANI.md) nejpozději v polední přestávce 2. dne.
5. Co nejde, hlaste hned. Kdo je hotový, pomůže sousedovi.

V TERMINÁLU

```
python over_prostredi.py --bez-codexu
# … 0 chyb. PŘIPRAVENO NA KURZ

git clone https://github.com/poli-cz/sample-service-codex-kurz sample-service
cd sample-service
python scripts/check_all.py debug
# 100% tests passed out of 49
python scripts/check_all.py asan tidy

codex --sandbox read-only "Vypiš, jakými
  příkazy se tento projekt staví a testuje.
  Nic neměň."
```

Hotovo, když: skript hlásí 0 chyb, debug končí 49 zelenými testy, asan a tidy projdou a po dotazu Codexu je git status čistý.

10 min

ictPRO

První úloha: agent jen čte

Plán, odkazy na soubor a funkci, přiznaná nejistota

1 C0 Kontrola prostředí | 2 C1 První úloha jen pro čtení

CO UDĚLAT

1. V kořeni repa: codex --sandbox read-only (ve VS Code přepněte rozšíření na režim jen pro čtení).
2. Vložte zadání vpravo, celé a beze změny.
3. Sledujte, co agent otevřel, co hledal a kolik kroků potřeboval.
4. Porovnejte odpověď s README.md a docs/architecture.md.
5. Zkrácená varianta: jen otázky 1 a 2.

ZADÁNÍ PRO CODEX

```
Jsi v C++ repozitáři, který neznáš. Nic neměň a nespouštěj build. Než
začneš číst, napiš plán: co a v jakém pořadí projdeš.

Pak odpověz, vždy s odkazem na soubor a funkci:
1. Popiš cestu jednoho vzorku od příkazu SUBMIT až po řádek result na
   výstupu: funkce v pořadí, jak se volají.
2. Jakým směrem jdou závislosti mezi knihovnami labcore, labio, labsched
   a apps? Ověř to podle #include a CMakeLists.txt, ne podle dokumentace.
   Najdeš nějaký #include, který jde proti směru?
3. Kde je fronta vzorků a kde stojan? Jaké datové struktury používají
   a kde se nastavuje jejich kapacita?
4. Které příkazy služba přijímá? Sedí to s tabulkou v README.md?
5. Uveď tři místa, kde si nejsi jistý, a proč.
```

Hotovo, když: máte odpovědi na pět otázek s odkazy, víte o aspoň jednom místě, kde agent odhadoval, a git status je čistý.

15 min

Co měl agent najít

Knihovny sample-service a směr závislostí

apps/sample-service
spustitelný program, řádkový protokol příkazů
`main.cpp` `command_processor.cpp`

↓ závisí na

src/labsched
simulovaný přístroj, stojan, plánovač
`scheduler.cpp` `rack.cpp` `instrument.cpp`

↓ závisí na

src/labio
řádky firmwaru, JSON, konfigurace, CSV
legacy parser v C stylu
`result_parser.cpp` `legacy_result_parser.cpp` `json_codec.cpp` `config.cpp` `csv_report.cpp`

↓ závisí na

src/labcore
doména, jen standardní knihovna
sem patří LAB-201
`sample_queue.cpp` `calibration.cpp` `statistics.cpp` `sample.cpp` `measurement.cpp`

Kontrola odpovědi

- Směr: apps → labsched → labio → labcore
- labcore smí jen standardní knihovnu
- Měl odkazy na soubory a řádky?
- Kde hádal? Typicky: proč je parser v C, co dělá příznak OK
- Vlastní kód: 3 627 řádků a 49 testů

Odpověď s odkazy na řádky se dá zkontrolovat. Odpověď bez nich je jen věrohodná.

ictPRO

BLOK 2 ZE 7

Jak coding agent uvažuje a jak mu zadávat

02

Co v zadání chybí, agent rozhodne odhadem. Často tiše.

90 MINUT · C2

Postup kurzu

ictPRO

Dvě dráhy stejné úlohy

Obě skončí zeleně. Jen jedna má pravdu.

Slabé zadání

Rozhraní
vymyslí vlastní

→

Chování
hodnotí jen poslední bod

→

Testy
potvrdí vlastní kód

→

Zelená
build i testy projdou

→

Jiná úloha, hotová
testy tiketu nejdou ani přeložit

Dobré zadání

Hrany
rozhodnuté v zadání

→

Testy napřed
z hran v tiketu

→

Kód
dokud testy neprojdou

→

Kontrola
debug a tidy

→

Hotovo, které platí
nebo jasná otázka zpátky: „> nebo ≥?“

Zelená znamená „odpovídá testům“. Jestli testy odpovídají požadavku, rozhodujete vy.

Jak model vidí text: tokeny

Model nečte slova ani znaky, ale tokeny

Angličtina The control value was rejected

`The` `·control` `·value` `·was` `·rejected` — **5 tokenů**

Čeština Kontrolní vzorek byl zamítnut

`Kont` `rol` `ní` `·vz` `orek` `·byl` `·zam` `í` `tn` `ut` — **10 tokenů**

C++ `auto& victim = rack_.lowest_priority_slot();`

`auto` `&` `·victim` `·=` `·rack` `_.` `lowest` `_priority` `_slot` `();` — **10 tokenů**

Skutečné rozdělení tokenizérem o200k_base (knihovna tiktoken, 26. 10. 2026). Tokenizér modelů v Codexu se může lišit.

Platíte za tokeny

Za všechno, co model čte i píše, v každém kroku smyčky.

Okno se měří v tokenech

Kontext není počet souborů, ale počet tokenů.

Čeština stojí dvojnásobek

Stejná věta: 5 tokenů anglicky, 10 česky. AGENTS.md klidně anglicky.

Token je jednotka čtení, myšlení i účtování.

Model předpovídá další token, i když neví

Vždy něco odpoví. „Nevím“ je málokdy nejpravděpodobnější.

1 · ZADÁNÍ SE ROZDĚLÍ NA TOKENY

`double` `·z` `·=` `·lab` `::` `stats` `::` `?`

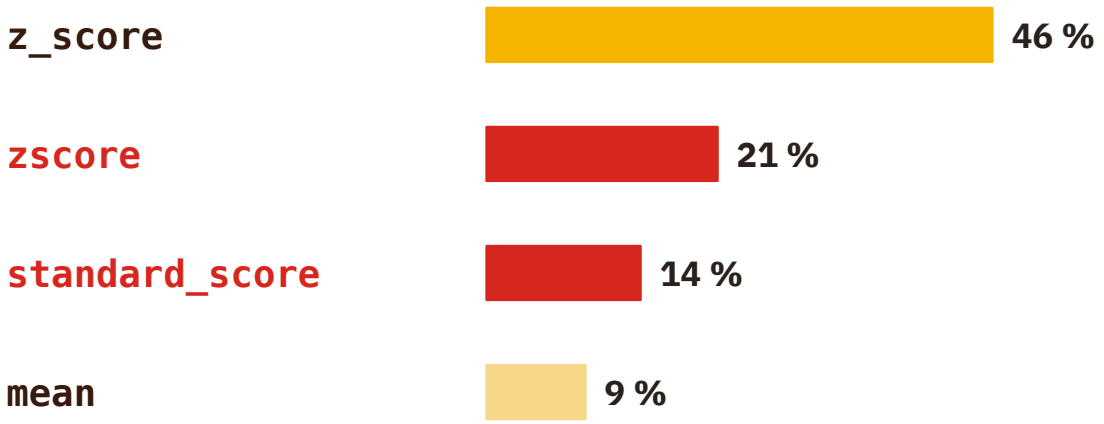
2 · TRANSFORMER POROVNÁ KAŽDÝ TOKEN S KAŽDÝM

MODEL

Desítky vrstev pozornosti. Žádná databáze, žádné vyhledávání v repozitáři. Jen váhy naladěné tréninkem na obrovském množství kódu.

Zdroj: ilustrativní čísla; v lab::stats existují z_score i mean

3 · PRAVDĚPODOBNOST KAŽDÉHO MOŽNÉHO DALŠÍHO TOKENU



Červeně: věrohodné, ale v sample-service neexistuje.

Plynulost není pravda. Skutečné API v kontextu halucinace sníží, ale nevyloučí.

Reasoning model: nejdřív uvažuje, pak jedná

Před odpovědí generuje tokeny uvažování. Kolik jich bude, určuje úsilí.

Úsilí	Popis v katalogu CLI 0.163.0 (přeloženo)	Kdy sáhnout
low	rychlé odpovědi s lehčím uvažováním	výchozí u gpt-6.1-sol; úloha se zadáním a testy
medium	vyvážená rychlost a hloubka pro každodenní úlohy	výchozí u gpt-6-sol a gpt-6-luna
high	větší hloubka pro složité problémy	plán, návrh, nejasná příčina chyby
xhigh	mimořádná hloubka pro složité problémy	výjimečně, když high nestačí
max	nejvyšší hloubka pro nejtěžší problémy	ne na rutinu: stojí čas a tokeny
ultra	nejvyšší hloubka s automatickým delegováním úloh	spouští pomocné agenty (zítra, blok 5)

Víc uvažování stojí čas a tokeny. Zapínejte ho na plán a ladění, ne na každý krok.

Trénink a použití

Proč model nezná vaše API ani verzi kompilátoru

Trénink: jednou

- Měsíce výpočtů, jednou za čas
- Znalosti zmrazené k datu
- Váš kód ani vaše pravidla tam nejsou
- Nová verze knihovny: model o ní neví

Použití: při každém zadání

- Zadání, AGENTS.md, soubory, výstupy příkazů
- Jediné místo, kudy přichází nová znalost
- Na tuto část sahá Codex i vy
- Co v kontextu není, model doplní odhadem

Model zná C++ obecně, sample-service vůbec. Co o něm ví, přišlo v kontextu.

Kontextové okno: pracovní paměť

Je to RAM, ne dlouhodobá paměť

272 tis.

tokenů se vejde do okna modelů v Codex CLI 0.163.0

30 tis.

tokenů má vlastní kód sample-service (3 627 řádků)

4,7 tis.

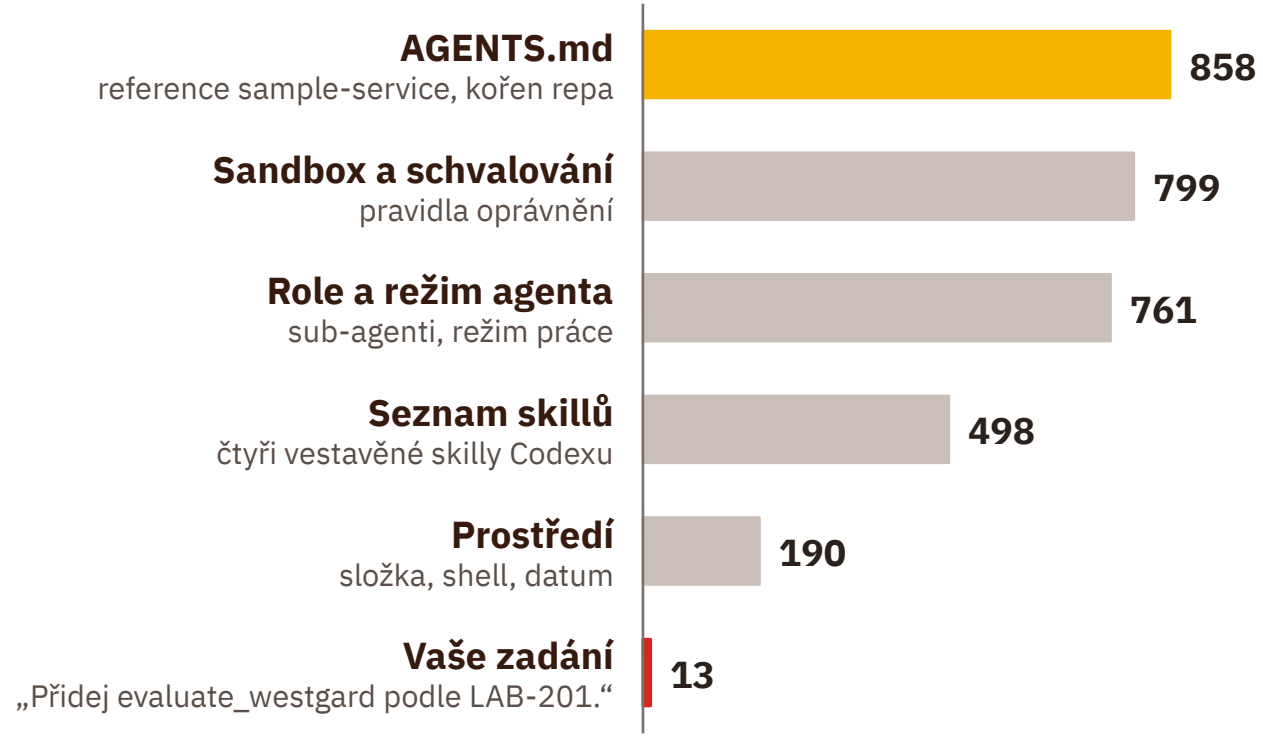
tokenů přidá jeden výpis buildu z čistého stavu

Co se do okna nevejde nebo z něj vypadne, pro model neexistuje.

Kolik z toho, co model čte, je vaše zadání

Vstup, který Codex 0.163.0 složí kolem jedné věty, ještě před prvním souborem

TOKENY V PRVNÍ ZPRÁVĚ (BEZ SYSTÉMOVÉHO PROMPTU A POPISU NÁSTROJŮ)



Co z toho plyne

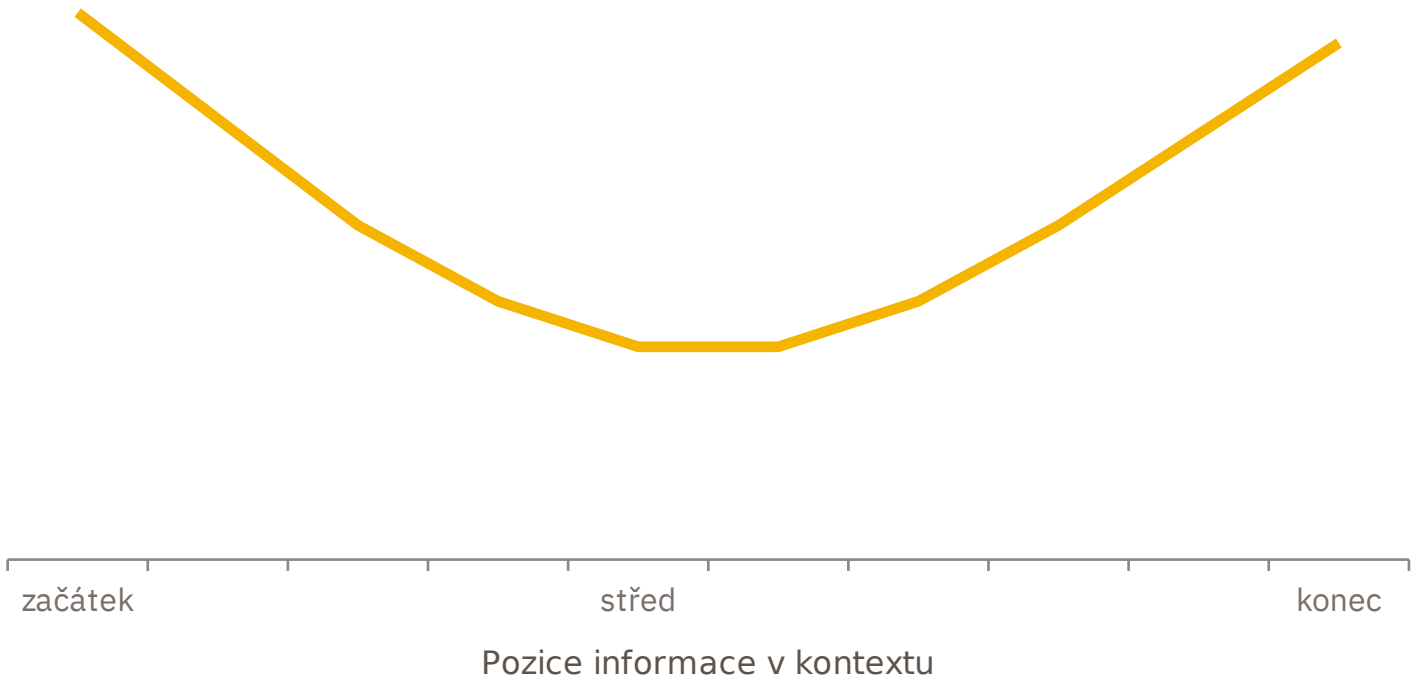
- Zadání je 13 tokenů z 3 119
- AGENTS.md jde do každé zprávy: krátký a konkrétní
- Každý skill, který si nainstalujete, přidá svůj popis do každé zprávy
- Pak přibývají soubory a výstupy příkazů, desítky tisíc tokenů

Zdroj: Měřeno 26. 10. 2026: codex debug prompt-input, CLI 0.163.0, prázdný HOME i CODEX_HOME, větev reference/agents-md, tokeny o200k_base

Model čte hlavně to, co jste nenapsali vy. Proto záleží na AGENTS.md a na tom, co agent najde.

Ztracené uprostřed

Jak dobře model použije informaci podle toho, kde v kontextu leží



Co z toho plyne

- Začátek a konec kontextu model využije nejlépe
- Pravidla patří na začátek (AGENTS.md) nebo do posledního zadání
- Dlouhý log buildu odsune podstatnou chybu do středu

Zdroj: Schematicky podle Liu a kol.: Lost in the Middle (2023)

Dlouhá session neškodí jen ceně, ale i přesnosti.

Rozklad úlohy: LAB-201 v šesti krocích

Tak by měl vypadat plán, který vám agent ukáže, než začne psát

01

Hlavička

Violation a deklarace funkce

02

Testy

případy z tiketu, červená

03

Jedna hodnota

1-2s, 1-3s, chybná SD

04

Dvě po sobě

2-2s a R-4s

05

Řady

4-1s a 10-x

06

Kontrola

debug, tidy, shrnutí

Každý krok má vlastní test. Když agent zabloudí, víte přesně kde.

Jak agent hledá v kódu

Skutečné výstupy stejných příkazů nad sample-service

```
$ rg -n "z_score" src
src/labcore/include/labcore/statistics.hpp:30:
  [[nodiscard]] double z_score(double value, ...
src/labcore/src/statistics.cpp:68:
  double z_score(double value, double target_mean, ...

$ rg -n "C\|" src/labio
src/labio/legacy/legacy_result_parser.cpp:6:
  *   C|pos|control lot|assay|signal|flags|timestamp

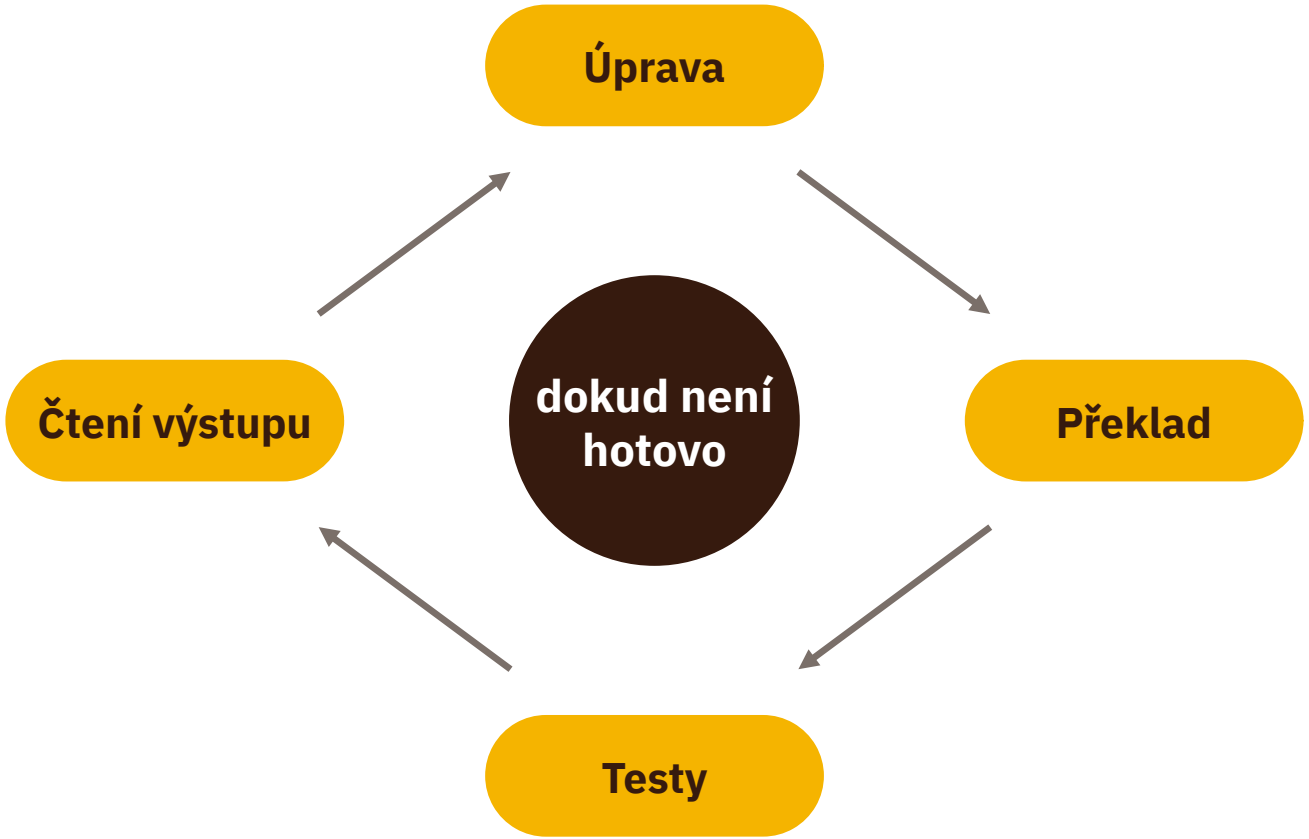
$ rg -n "add_library" src
src/labcore/CMakeLists.txt:1:add_library(labcore
src/labio/CMakeLists.txt:1:add_library(labio
src/labsched/CMakeLists.txt:1:add_library(labsched
```

- **Hledá jména**
  rg je rychlejší než číst všechno
- **Čte hlavičky**
  veřejné API, ne celou implementaci
- **Čte CMakeLists**
  který cíl, která knihovna, které testy
- **Build jako sonda**
  chyba překladače ukáže, co chybí

rg najde z_score za sekundu. Proč vrací pro SD 0 nulu, v kódu nenajde.

Iterace a sebekontrola

Smyčka končí, když agent usoudí, že je hotovo



- **Hotovo ze zadání**
  spustitelné: preset, test, tidy
- **Bez kritéria hotovo**
  končí, když to vypadá hotově
- **Sebekontrola**
  „uveď, co jsi ověřil a co ne“
- **Pozor**
  test upravený tak, aby prošel

Agent se ověřuje jen tím, co mu dáte do ruky. Spustitelné kritérium hotovo je nejlevnější kontrola.

Halucinace naživo: co řekne GCC 12

Čtyři věrohodné řádky, čtyři skutečné chyby překladu

CO AGENT NAPSAL

```
// 1 neexistující API
lab::stats::stddev(v);

// 2 jiný standard (C++23)
std::ranges::fold_left(v, 0.0,
                       std::plus<>{});

// 3 knihovna, kterou GCC 12 nemá
#include <format>

// 4 špatná hlavička
#include "labcore/qc_rules.h"
```

CO ŘEKL GCC 12.2 (sample-service, -std=c++20)

```
error: 'stddev' is not a member of
       'lab::stats'

error: 'fold_left' is not a member of
       'std::ranges'


fatal error: format: No such file or
             directory

fatal error: labcore/qc_rules.h: No such
             file or directory
```

To jsou levné chyby: build je chytí hned. Agent je opraví sám, když build smí spustit.

Projde to?

Domyšlené chování šablony, ne neexistující API

```
// labcore: průměr kontrolních hodnot
double mean_of(const std::vector<double>& v) {
  return std::accumulate(v.begin(), v.end(), 0)
         / static_cast<double>(v.size());
}

// vstup: 99.6, 100.4, 100.9, 98.7
// správný průměr: 99.90
```

KVÍZ

Projekt překládá s -Wall -Wextra -Wconversion -Werror. Co se stane?

A Nepřeloží se, -Wconversion to chytí.

B Přeloží se a vrátí 99,90.

C Přeloží se a vrátí 99,25.

D Přeloží se, ale chytí to clang-tidy.

Hlasujte rukou. Odpověď je na dalším slidu a není jen jedna.

Odpověď: přeloží se, chytí to až clang-tidy

Skutečný výstup, stejné přepínače jako v sample-service

```
$ g++ -std=c++20 $FLAGS acc.cpp
  # FLAGS: -Wall -Wextra -Wconversion
  #        -Wsign-conversion -Werror
$ ./a.out
99.25                    # správně 99.90

$ clang-tidy acc.cpp
error: folding type 'double' into type 'int'
  might result in loss of precision
  [bugprone-fold-init-type]
```

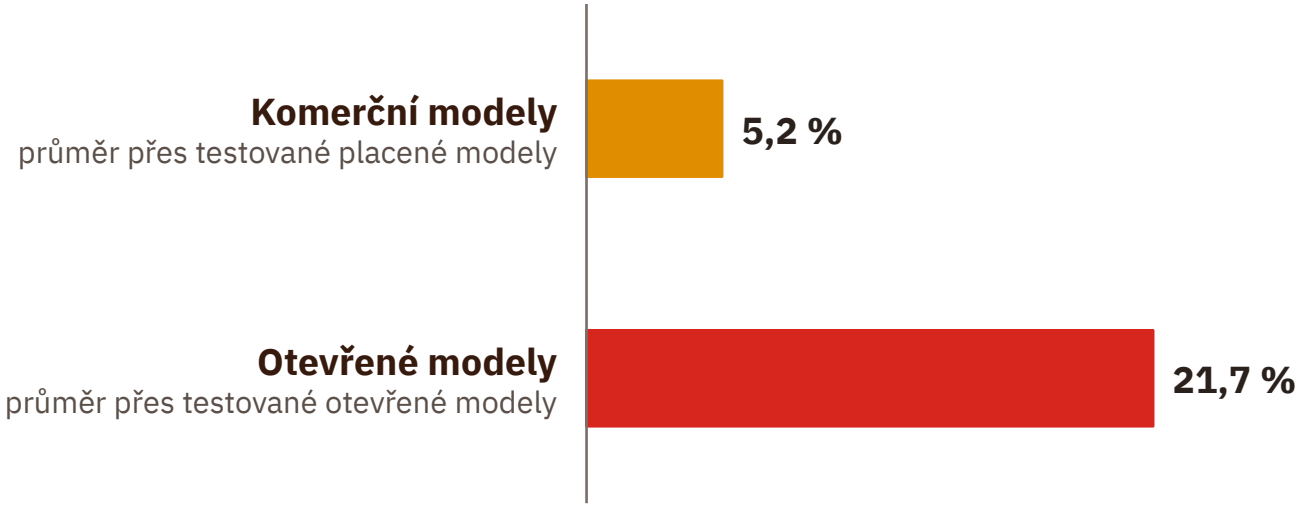
- **Šablona to „ví“**
  typ součtu je typ počáteční hodnoty
- **Překladač mlčí**
  konverze je uvnitř standardní knihovny
- **Test s hodnotou**
  chytí to, když hodnotu určil člověk
- **clang-tidy**
  proto patří preset tidy do definice hotovo

Nejdražší halucinace se přeloží. Chytí ji test s hodnotou od člověka nebo statická analýza.

Vymyšlený balíček: halucinace, kterou build nechytí

Podíl doporučených balíčků, které vůbec neexistovaly

PRŮMĚRNÝ PODÍL NEEXISTUJÍCÍCH BALÍČKŮ VE VYGENEROVANÉM KÓDU



Proč je nejdražší

- Build ani testy nic nehlásí, dokud balíček chybí
- Útočník si vymyšlené jméno zaregistruje a čeká (slopsquatting)
- Studie měřila Python a JavaScript; vcpkg a Conan ne
- Novou závislost schvaluje člověk a ověří její zdroj

Zdroj: Spracklen a kol.: We Have a Package for You!, USENIX Security 2025

Překladač chytí vymyšlené API. Vymyšlený balíček chytí jen člověk.

Zadání pro agenta není prompt do chatu

Je to specifikace práce na desítky kroků

Prompt do chatu

- Otázka, odpověď hned
- Kontext je jen to, co vložíte
- Hotovo posoudíte okem
- Nejasnost vyřeší další otázka

Zadání pro agenta

- Práce na desítky kroků bez vás
- Kontext si agent hledá v repu sám
- Hotovo musí být spustitelné: preset, test
- Nejasnost agent vyplní odhadem, často tiše

Čím déle agent pracuje bez vás, tím víc rozhodnutí musí být v zadání.

Anatomie zadání pro C++

Pět částí, každá jedna až dvě věty

```
Cíl: přidej lab::qc::evaluate_westgard podle LAB-201.

Výstup: qc_rules.hpp, qc_rules.cpp a testy v
GoogleTest.

Rozsah: jen src/labcore a tests/labcore, jinde nic
neměň.

Omezení: C++20, GCC 12 a Clang 16, jen standardní
knihovna.

Hotovo: python scripts/check_all.py debug tidy
projde.
```

CÍL

VÝSTUP

ROZSAH

OMEZENÍ

HOTOVO

PROČ TO FUNGUJE

- Rozsah zabrání „úklidu“ v labio a labsched
- Omezení zabrání C++23 a novým závislostem
- Hotovo dává agentovi důvod spustit build a tidy
- Šablona se dá použít znovu

Napsat ho trvá minutu. Ušetří tři iterace.

Každá úloha se láme jinde

Pět částí je kostra. Kterou větu nesmíte vynechat, určuje typ úlohy.

Typ úlohy	Co rozhoduje	Věta do zadání	Typická past
Oprava chyby	reprodukce, ne popis	„Nejdřív test, který chybu ukáže a selže. Pak oprav.“	opraví příznak, chyba se vrátí jinde
Nová funkce v modulu	vzor a hranice	„Jako SampleQueue. Mimo src/labcore nic neměň.“	vlastní styl, druhá kopie existující funkce
Refaktoring	chování se nemění	„Veřejné API a výstupy zůstávají. Testy neupravuj.“	změna logiky schovaná v přesunu kódu
Orientace v kódu	jen čtení, s odkazy	„Nic neměň. U každého kroku soubor a řádek.“	věrohodný popis, který si domyslel
Doménový výpočet	hodnoty určuje člověk	„Hodnoty v testech jsou závazné. Při rozporu se zeptej.“	„opraví“ test podle svého kódu

Před zadáním si řekněte, o jaký typ úlohy jde. Podle toho víte, kde agent nejspíš selže.

LAB-201: slabé a dobré zadání

Výňatky ze zadání cvičení C2

SLABÉ ZADÁNÍ

```
Přidej do labcore vyhodnocení
Westgardových pravidel (1-2s, 1-3s,
2-2s, R-4s, 4-1s, 10-x) pro kontrolní
vzorky.
```

DOSTANETE

- Vlastní rozhraní, ne to z tiketu
- Hrany a chování podle modelu
- Změny i mimo rozsah

DOBRÉ ZADÁNÍ

```
Cíl: přidej do knihovny labcore […]
(tiket LAB-201). Rozhraní: […]
„Za limitem“ znamená ostře větší […]
1-2s se hlásí vždy, i vedle ostatních.
Postup: nejdřív testy […] Spusť je
a ukaž, že padají. Hotovo, když:
python scripts/check_all.py debug tidy
projde: […] tidy nic nehlásí.
```

DOSTANETE

- Rozhraní a hrany podle tiketu
- Testy z tiketu, ne z implementace
- Hotovo ověřené příkazem, ne pocitem

Dobré zadání není delší z principu. Je konkrétní tam, kde slabé mlží.

Jedna věta, dvě sady testů

Totéž vyhodnocení, jen jiný výklad slova „za mezí“ (průměr 100, SD 10)

Kontrolní hodnoty	„Za mezí“ = ostře větší	„Za mezí“ = větší nebo rovno
120 (přesně +2 SD)	nic	1-2s na indexu 0
130 (přesně +3 SD)	1-2s na indexu 0	1-3s a 1-2s na indexu 0
110, 110, 110, 110	nic	4-1s na indexu 3
120, 120	nic	2-2s na indexu 1, dvakrát 1-2s
101 až 110 (deset hodnot)	10-x na indexu 9	10-x na indexu 9, stejné

Rozdíl ukáže jen hodnota přesně na mezi. Ten řádek do tabulky dává člověk.

CVIČENÍ C2 · 40 MINUT

Slabé a dobré zadání

Ve dvojicích: jeden slabé, druhý dobré, po 15 minutách si výsledky ukážete

CÍL

Zjistit, co všechno agent u slabého zadání rozhodl za vás. Stejná úloha dvakrát: jako jedna věta a se strukturou cíl, výstup, chování, rozsah, omezení, postup a hotovo.

`cviceni/02-zadani`

POSTUP

1. Čistý start před každým pokusem
2. Slabé zadání, sloupec „slabé“ v mřížce
3. Čistý start, nová session, dobré zadání
4. Porovnání ve dvojici podle mřížky (10 min)
5. Kontrola tabulkou: 120, 130, 101 až 109 a 100

ODEVZDÁTE

Vyplněnou hodnoticí mřížku pro obě zadání a seznam rozhodnutí, která agent u slabého zadání udělal sám.

HOTOVO, KDYŽ

víte, jestli agent u slabého zadání spustil testy a tidy a jestli to řekl ve shrnutí.

ictPRO

Slabé zadání

Jedna věta, bez tiketu, v čistém repozitáři

CO UDĚLAT

1. Čistý start po jednom: git switch main · git restore . · git clean -fd
2. Nová session: codex --sandbox workspace-write
3. Vložte přesně zadání vpravo, nic dalšího.
4. Až agent skončí: git status, git diff --stat, přečtěte jeho shrnutí.
5. Vyplňte sloupec „slabé“ v hodnotici-mrizka.md.

ZADÁNÍ PRO CODEX

```
Přidej do labcore vyhodnocení
Westgardových pravidel (1-2s, 1-3s,
2-2s, R-4s, 4-1s, 10-x) pro kontrolní
vzorky.
```

Hotovo, když: máte git diff --stat, shrnutí agenta a vyplněný sloupec „slabé“.

15 min

ictPRO

Dobré zadání

Čistý start, nová session, celé zadání ze ZADANI.md

CO UDĚLAT

1. Čistý start (krok 1), nová session, ne resume.
2. Vložte celé dobré zadání ze ZADANI.md, vpravo je výňatek.
3. Agent má nejdřív ukázat padající testy.
4. Vyplňte sloupec „dobré“ v mřížce.

ZADÁNÍ (VÝŇATEK, VLOŽTE CELÉ ZE ZADANI.md)

```
Cíl: přidej do knihovny labcore vyhodnocení Westgardových pravidel pro
kontrolní vzorky (tiket LAB-201).
Rozhraní: nový hlavičkový soubor src/labcore/include/labcore/qc_rules.hpp,
implementace v src/labcore/src/qc_rules.cpp: […]
Chování […]: „Za limitem“ znamená ostře větší než horní limit, nebo
ostře menší než dolní limit. Hodnota přesně na limitu ho nepřekračuje; […]
target_sd <= 0 vyhodí std::invalid_argument. Prázdný vstup vrátí
prázdný vektor.
Rozsah: jen nové soubory qc_rules.hpp, qc_rules.cpp, […] nic jiného neměň.
Technická omezení: C++20, jen standardní knihovna, žádné nové závislosti, […]
Postup: nejdřív testy v GoogleTest: […] Spusť je a ukaž, že padají.
Pak implementace.
Hotovo, když: python scripts/check_all.py debug tidy projde: testy
zelené, tidy nic nehlásí. Na konci shrň, co jsi změnil, které příkazy
jsi spustil s jakým výsledkem a co jsi neověřil.
```

Hotovo, když: testy prošly, agent vypsal, co spustil a co neověřil, a máte vyplněný sloupec „dobré“.

15 min

Hodnoticí mřížka

Body: 0 = ne, 1 = částečně, 2 = ano. Do poznámky, co jste konkrétně viděli.

#	Kritérium	Slabé	Dobré
1	Rozhraní: hlavička, jméno funkce, typy a jmenný prostor podle tiketu	0 · 1 · 2	0 · 1 · 2
2	Hranice: hodnota přesně na limitu 1, 2 a 3 SD se chová podle zadání	0 · 1 · 2	0 · 1 · 2
3	Hlášení: pravidlo i index; 1-2s i vedle ostatních; hlásí se každé okno	0 · 1 · 2	0 · 1 · 2
4	Neplatný vstup: SD 0 nebo záporná vyhodí std::invalid_argument	0 · 1 · 2	0 · 1 · 2
5	Testy existují a testují požadavek, ne jen to, co kód zrovna dělá	0 · 1 · 2	0 · 1 · 2
6	Testy napřed: agent ukázal, že před implementací padají	0 · 1 · 2	0 · 1 · 2
7	Ověření: agent sám spustil debug i tidy a obojí prošlo	0 · 1 · 2	0 · 1 · 2
8	Rozsah: jen to, co měl (žádná dokumentace, CMake ani soubory navíc)	0 · 1 · 2	0 · 1 · 2
9	Shrnutí: spuštěné příkazy s výsledkem a co neověřil	0 · 1 · 2	0 · 1 · 2
	Součet (max. 18)	…	…

Nehodnotíme agenta. Hodnotíme, kolik rozhodnutí jste mu nechali.

Co agent domyslel

1. Co agent u slabého zadání rozhodl sám?
2. Která věta dobrého zadání změnila nejvíc?
3. Zeptal se vás agent na něco? Na co?



Co Codex skutečně udělal: zkušební běhy

Stejné zadání jako v C2, dva běhy každého, čistý repozitář

	Slabé zadání (2 běhy)	Dobré zadání (2 běhy)
Změněné soubory	6 a 6, včetně docs/architecture.md	5 a 5: labcore, testy, CMake
„Za mezí“	ostře větší v obou bězích	ostře větší, jak říká zadání
Rozhraní z tiketu	ne: vlastní typ ControlResult, hodnotí jen poslední hodnotu	ano: lab::qc::evaluate_westgard
debug a tidy	debug v obou bězích, tidy jednou selhal	oba v obou bězích
Akceptační testy (14)	nejdou přeložit: jiné rozhraní	14 z 14, v obou bězích

Slabé zadání nedalo špatný kód. Dalo dobře otestovaný kód pro jinou úlohu.

Iterativní zpřesňování

Když agent odbočí: neopravujte výsledek, opravte zadání

01

Zastavit

Esc, nic dalšího neschvalovat

02

Zjistit proč

chyběl kontext, plán, nebo kritérium hotovo?

03

Opravit zadání

přidat větu, která chyběla

04

Nová session

se souhrnem, ne „ne, takhle ne“ podesáté

Po dvou neúspěšných opravách nepokračujte. Vraťte se a opravte zadání.

Čtyři triky, které zlepší výsledek

Až zvládnete pět částí zadání

Ukažte vzor

„Udělej to jako SampleQueue a její testy.“ Existující soubor je lepší než popis.

Nechte se ptát

„Než začneš, polož mi otázky k nejasnostem.“ Odhalí díry v zadání.

Rozdělte úlohu

Orientace, plán, testy, implementace jako samostatné kroky.

Řekněte, na co nesahat

„Testy a CMakePresets.json neměň.“ Zákaz je kratší než vysvětlení.

Nejčastější chyby při práci s Codexem

A co s nimi

1	**Schválený plán, který nikdo nečetl**	Agent udělá přesně to, co jste neviděli.	*plán číst, pak zápis*
2	**Dvě úlohy v jedné session**	Kontext první úlohy plete druhou, rozsahy se míchají.	*jedna úloha, jedna session*
3	**Dlouhá session**	Starší pokyny se ztrácejí uprostřed kontextu.	*/compact nebo /new*
4	**Velký diff najednou**	Revize je formalita, nikdo ho nepřečte.	*malé kroky*
5	**Věřit shrnutí**	„Testy prošly“ bez výstupu příkazu.	*chtějte log*

Většina chyb se dá opravit jednou větou v zadání. Opakované věty patří do AGENTS.md.

BLOK 3 ZE 7

Instrukce a kontext projektu

03

Co opakujete v každém zadání, patří jednou do repozitáře

105 MINUT · C3 A C3b

Postup kurzu

Agent každé ráno začíná od nuly

Nepamatuje si včerejší session ani vaše zvyklosti

0

paměti mezi novými session

- **Pondělí: vysvětlíte, že se staví přes presety**
- **Úterý: nová session, agent zase staví do vlastního adresáře**
- **codex resume vrátí historii jednoho vlákna, ne znalost projektu**
- **Co má platit pořád, musí být v souboru, který se načte sám**

Co jste v C2 museli psát do každého zadání, patří do souboru v repozitáři.

Co agent najde sám a co mu musí říct tým

AGENTS.md neopakuje kód. Obsahuje to, co ví tým.

Najde sám

Strukturu a jména
rg, hlavičky, CMakeLists

Jak se projekt staví
README, CMakePresets.json

Styl kódu
z okolních souborů

Směr závislostí
z #include a CMake, jako v C1

Musí mu říct tým

Co je hotovo
debug a tidy; u paměti a životnosti objektů i asan

Doménová pravidla
SD ≤ 0 je chyba; z_score se pro kontrolu kvality nepoužívá

Co se nesmí změnit
struktura LEGACY_RESULT; žádné NOLINT ani -Wno-*

Závislosti
nové jen po schválení člověkem

Do AGENTS.md patří pravý sloupec. Levý si agent přečte sám.

Co patří do AGENTS.md u C++ projektu

Soubor v kořeni repozitáře, Codex ho čte automaticky

```
## Build: python scripts/check_all.py debug

## Smyčka: --target <lib>_tests, ctest -R <Suite>

## Závislosti: find_package, jinak third_party/
(neupravovat)

## Konvence: C++20, snake_case, -Werror bez výjimek

## Nesahat: CMakePresets.json, .clang-tidy, legacy/

## Hotovo: debug a tidy projdou, testy, popis MR
```

BUILD

SMYČKA

ZÁVISLOSTI

KONVENCE

NESAHAT

HOTOVO

PROČ TO FUNGUJE

- Spustitelné příkazy, ne popisy
- Pravidla, která agent jinak poruší
- Krátké: tokeny za něj platíte v každém tahu
- Kostru vygeneruje /init, pak zkrátit

Dobrý AGENTS.md je krátký a spustitelný. Esej nikdo nečte, ani agent.

Vrstvy instrukcí: co Codex skutečně načte

Systém, osobní soubor, kořen repa, modul, zadání

SOUBORY S INSTRUKCEMI

- `~/.codex/` **AGENTS.md** — osobní
- `sample-service/` **AGENTS.md** — kořen, 3,2 KiB
  - `src/labio/` **AGENTS.md** — modul, 1,4 KiB
    - `legacy/` **← Codex běží tady**
  - `src/labsched/` AGENTS.md — sourozenec: nenačte se
  - `.agents/skills/` — jen jméno a popis

Pořadí: systém Codexu, osobní, kořen, složky ke složce běhu, zadání

Bližší vyhrává: hlubší soubor je v kontextu později

Náhradní soubor: AGENTS.override.md nahradí AGENTS.md ve stejné složce

Načítá se při startu: po změně AGENTS.md nová session

CODEX DEBUG PROMPT-INPUT (NADPISY)

```
# AGENTS.md instructions for
#   …/sample-service/src/labio/legacy
<INSTRUCTIONS>
Odpovídej česky, kód a identifikátory anglicky.
--- project-doc ---
# AGENTS.md
## Project
## Build and test
## Dependencies
## Conventions
## Tests
## Do not touch without asking
## Done means
# AGENTS.md for src/labio
## Legacy firmware parser (`legacy/`)
## Reports and config
</INSTRUCTIONS>
```

Obecné výš, konkrétní níž. Agent dostane přesně ten výřez pravidel, kde pracuje.

32 KiB: strop pro součet projektových souborů

Pokusy v Codex CLI 0.163.0 (project_doc_max_bytes = 32768)

Pokus	Kořenový AGENTS.md	Modul src/labio	Co se načte
Reference sample-service	3,2 KiB	1,4 KiB	oba celé
Velký kořen	30 KiB	krátký	oba celé
Příliš velký kořen	40 KiB	krátký	kořen uříznutý na 32 KiB, modul vypadne
Projekt označený jako untrusted	libovolný	libovolný	jen osobní ~/.codex/AGENTS.md

Přebujelý kořen tiše vytlačí pravidla modulů. Codex přitom nic nehlásí.

Kam napsat které pravidlo

Šest otázek, v tomto pořadí

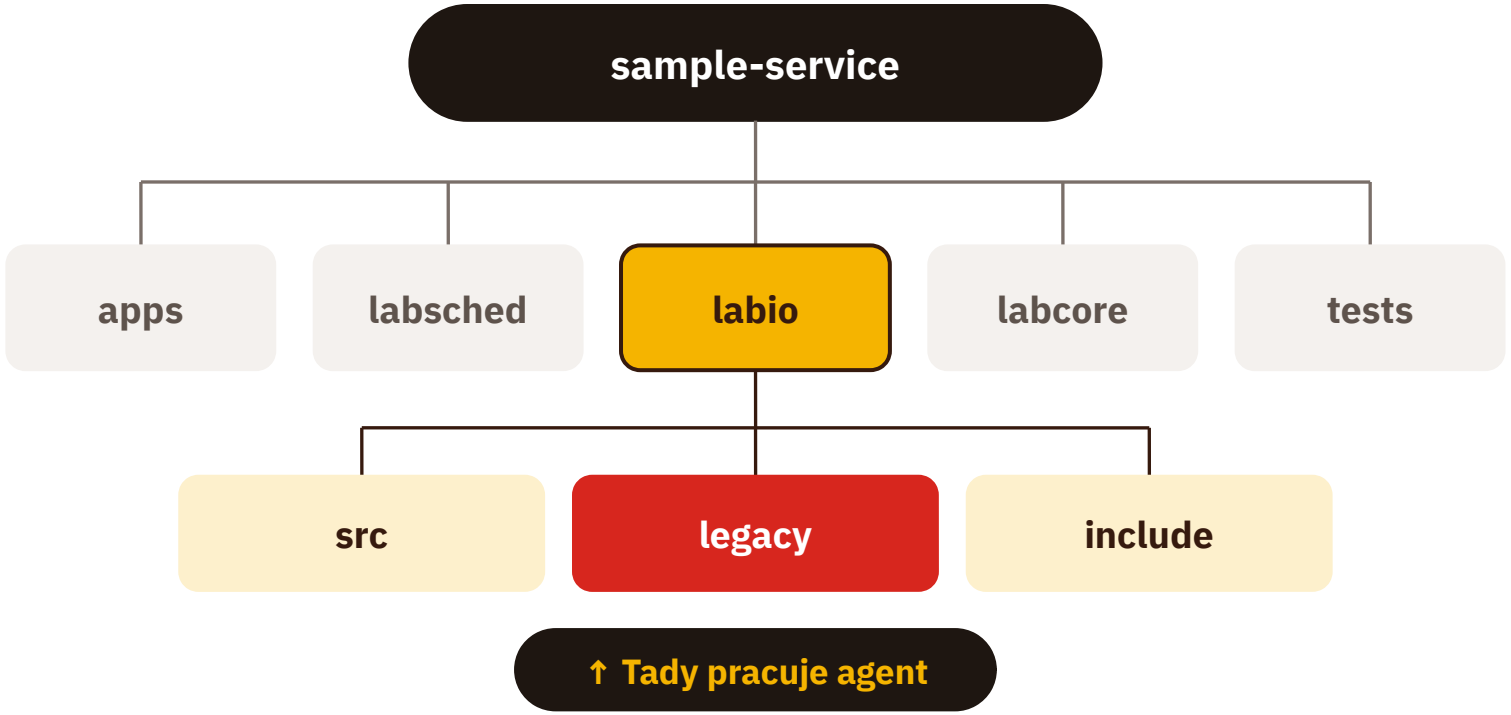
1. **Je to heslo, token nebo citlivá data?**
   - ano → **Ani do instrukcí, ani do repa**
     `tajemství do trezoru, data jen anonymizovaná`
   - ne ↓
2. **Platí to vždy pro celý repozitář?**
   - ano → **Kořenový AGENTS.md**
     `sample-service/AGENTS.md`
   - ne ↓
3. **Platí to jen pro jeden modul nebo složku?**
   - ano → **AGENTS.md v modulu**
     `src/labio/AGENTS.md`
   - ne ↓
4. **Je to postup, který se dělá jen někdy?**
   - ano → **Skill**
     `.agents/skills/<postup>/SKILL.md`
   - ne ↓
5. **Je to stav rozdělané práce?**
   - ano → **Soubor se stavem, ne instrukce**
     `HANDOFF.md, PROGRESS.md`
   - ne ↓
6. **Je to jen můj osobní zvyk?**
   - ano → **Osobní AGENTS.md**
     `~/.codex/AGENTS.md`
   - ne ↓

Nic z toho → **Do zadání**
`platí jen pro tuto úlohu`

Pravidlo na špatném místě buď chybí, když je potřeba, nebo ho platíte v každé zprávě.

Velká kódová báze: mapa a výřez

Celý repozitář se do kontextu nevejde. A nemusí.



- **Mapa modulů**
  v AGENTS.md: co který modul dělá, směr závislostí
- **Výřez úlohy**
  v zadání: „pracuj jen v src/labio/legacy“
- **Mimo kontext**
  build/, third_party/, vcpkg_installed/, generované soubory

Agent nepotřebuje znát celé repo. Potřebuje vědět, kde hledat a kde ne.

Velké závislosti mimo repozitář

Co musí agent vidět a jak mu to povolit

Zdroj	V sample-service	Co agent potřebuje	V sandboxu
Systémové balíčky	find_package: fmt, nlohmann_json, GTest	hlavičky v systému, číst smí	funguje, nic nestahuje
Kopie v repu	third_party/: co find_package nenajde	nic navíc, kopie neupravovat	funguje, nic nestahuje
vcpkg (manifest)	preset vcpkg, vcpkg.json, builtin-baseline	VCPKG_ROOT a binární cache	bez sítě nic nestáhne: cache předem
Conan	v sample-service není	profil a lokální cache	stejné: připravit předem
Sdílené SDK, binárky	-DCMAKE_PREFIX_PATH=/cesta	číst smí, zapisovat ne	cestu napsat do AGENTS.md

Agent smí číst mimo repo, ale nesmí stahovat. Závislosti připravte, cestu napište do AGENTS.md.

Sandbox naživo: co agent smí

Stejný sandbox, jaký Codex použije pro příkazy agenta

```
$ codex sandbox -c 'sandbox_mode="workspace-write"' \
    -- sh -c 'echo ok > inside.txt
              echo x >> .git/config
              echo y > .agents/skills/x.md
              curl -sS https://github.com'
sh: .git/config: Operation not permitted
sh: .agents/skills/x.md: Operation not permitted
curl: (6) Could not resolve host: github.com

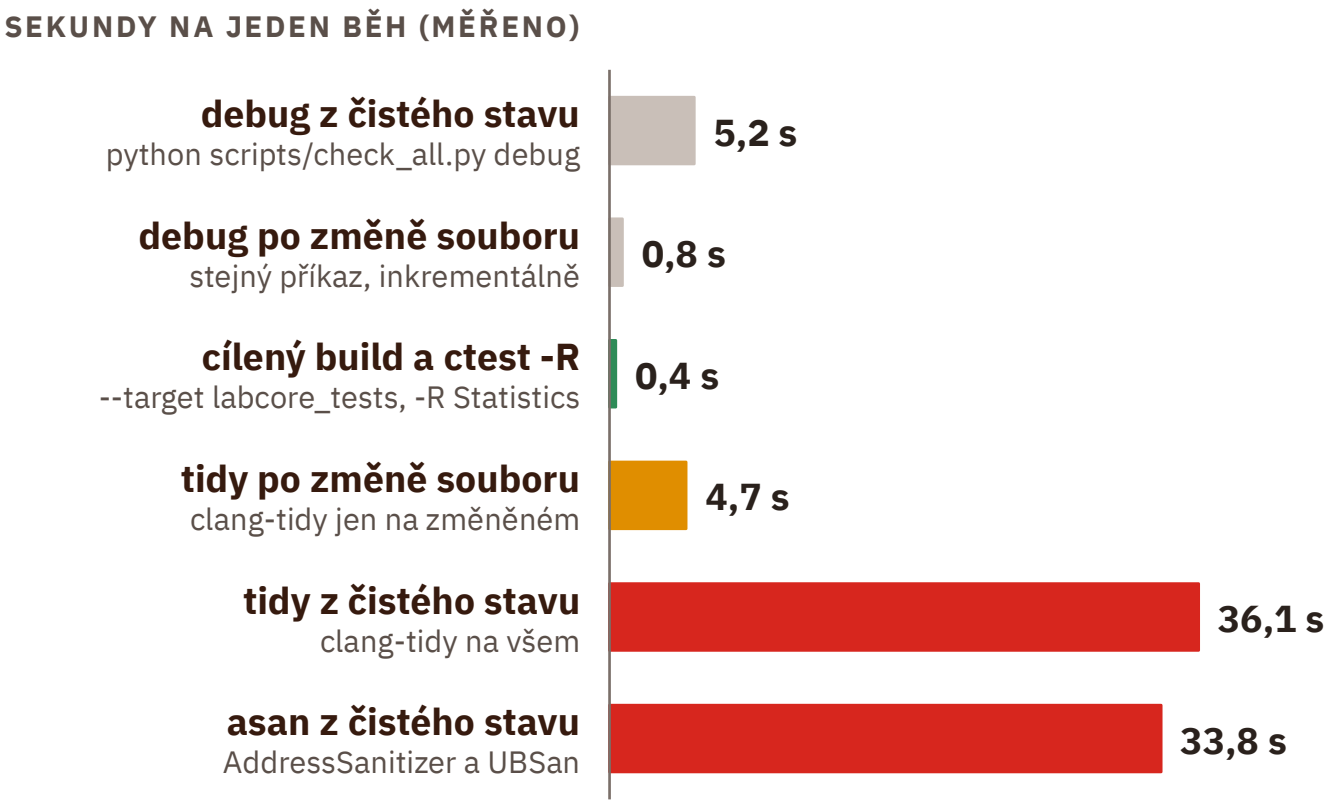
$ ls inside.txt
inside.txt
```

- **Zápis v repu**
  ano, v pracovní složce
- **.git, .agents, .codex**
  jen pro čtení i ve workspace-write
- **Síť**
  vypnutá: vcpkg ani git fetch nic nestáhnou
- **Výjimka**
  --add-dir přidá další zapisovatelnou složku

Co agent potřebuje stáhnout, připravte předem. Co potřebuje číst, popište v AGENTS.md.

Kolik stojí jedna kontrola

sample-service: presety, cílený build a filtrované testy



Do AGENTS.md

- Smyčka: --target <lib>_tests a ctest -R <Suite>
- Před „hotovo“: debug a tidy
- Změna paměti nebo kontejnerů: i asan
- Žádné vlastní build adresáře, presety staví do build/<preset>/

Zdroj: Měřeno 26. 10. 2026: sample-service v kontejneru z Dockerfile repa, Docker na Macu (10 jader), GCC 12.2, clang-tidy 14

Sekundová smyčka pro každý krok, drahé kontroly jednou před odevzdáním.

Instrukce se verzují jako kód

AGENTS.md v Gitu, změny přes merge request

gitlab.example.com/lab/sample-service/-/merge_requests/57

2 **!57 AGENTS.md: asan for memory and lifetime changes**

Open `agents/asan-rule → main` 1 commit · 1 soubor

1

AGENTS.md +3 −0

```
@@ -15,2 +15,5 @@ ## Build and test
 - Everything, one command: `python scripts/check_all.py debug`
 - Before you say a change is done, also run:
   `python scripts/check_all.py tidy`
+- Any change that touches memory ownership, containers or
+  lifetimes (rack, queue, scheduler, parser): also
+  `python scripts/check_all.py asan`
```

3 Approval required: code owner of AGENTS.md (vlastník instrukcí) Approve Merge 4

5

```
$ git pull   # pak nová session: AGENTS.md se načte jen při startu
```

Postup

1. **Změna** — AGENTS.md jako každý jiný soubor
2. **Merge request** — diff instrukcí, ne jen kódu
3. **Revize** — vlastník instrukcí schvaluje
4. **Merge** — všichni po git pull
5. **Nová session** — AGENTS.md se načte jen při startu

Neverzovaná instrukce je osobní zvyk. Verzovaná je pravidlo týmu.

Proč dva kolegové dostanou různý kód

Nekonzistentní výstupy jsou většinou nekonzistentní nastavení

1	**Jiné osobní instrukce**	~/.codex/AGENTS.md se liší člověk od člověka.	*osobně jen jazyk a zvyky*
2	**Jiný model a úsilí**	Každý má jiný config.toml.	*sdílený profil*
3	**Stará session**	AGENTS.md načtený před poslední změnou.	*nová session po pull*
4	**Různá zadání**	Každý píše po svém.	*skill nebo šablona zadání*
5	**Nic se nevynucuje**	Pravidla jen v hlavách.	*spravovaná konfigurace, zítra*

Stejné vstupy, podobné výstupy. Nedeterminismus modelu zbude, ten chytí testy.

Sdílený profil pro tým

Profily v2: jeden soubor na profil

```
# ~/.codex/tym.config.toml
model = "gpt-6.1-sol"
model_reasoning_effort = "medium"
sandbox_mode = "workspace-write"
approval_policy = "on-request"

[sandbox_workspace_write]
network_access = false

$ codex -p tym
```

- **Profil**
  soubor ~/.codex/<jméno>.config.toml
- **Spuštění**
  codex -p <jméno>
- **Pozor**
  staré [profiles.x] v config.toml už nefungují
- **Projekt**
  .codex/config.toml v repu, jen v důvěryhodném projektu

Stejný profil v celém týmu: stejný model, stejné úsilí, stejný sandbox.

CVIČENÍ C3 · 50 MINUT

AGENTS.md pro vzorový projekt

Napsat, vyzkoušet na malé úloze, porovnat před a po

CÍL

Napsat AGENTS.md s tím, co ví tým a co agent v kódu nenajde, a změřit, co změní: stejnou malou úlohu dát agentovi před a po.

`cviceni/03-agents-md`

POSTUP

1. Před: malá úloha bez AGENTS.md (10 min)
2. Napsat AGENTS.md (20 min)
3. Commit do vlastní větve
4. Po: nová session, stejná úloha (10 min)
5. Porovnat s reference/agents-md (5 min)

ODEVZDÁTE

AGENTS.md ve větvi c3-agents-md a záznam před a po: příkazy, tidy, chování pro SD 0, shrnutí.

HOTOVO, KDYŽ

umíte říct, které pravidlo z AGENTS.md změnilo chování agenta a které nezměnilo nic.

ictPRO

Před: malá úloha bez AGENTS.md

Zapisujte, co agent dělá, ne jen výsledek

CO UDĚLAT

1. Čistý stav po jednom: git switch main · git restore . · git clean -fd
2. Nová session: codex --sandbox workspace-write
3. Vložte malou úlohu vpravo, slovo od slova.
4. Zapište: příkazy, tidy ano či ne, co vrací pro SD 0 a pro 120, co napsal ve shrnutí.
5. Pak zase čistý stav.

ZADÁNÍ PRO CODEX

```
Přidej do labcore funkci
lab::qc::control_zone(double value,
  double target_mean, double target_sd),
která vrátí zónu kontrolní hodnoty jako
int: 0 uvnitř ±1 SD, 1 za 1 SD, 2 za 2 SD,
3 za 3 SD; kladné číslo nad průměrem,
záporné pod ním. Přidej testy a ověř,
že všechno projde.
```

Hotovo, když: máte zapsané příkazy agenta, výsledek tidy, chování pro SD 0 a pro 120 a jeho shrnutí.

10 min

ictPRO

Napsat AGENTS.md

Krátce, anglicky, každé pravidlo ověřitelné příkazem nebo v diffu

CO UDĚLAT

1. Obsah: seznam „Co ví tým“ v ZADANI.md. Rozhodněte, co z něj patří do AGENTS.md.
2. Pomůže kontrolni-seznam.md. Kostru může dát /init, pak ji seškrtejte.
3. Do souboru jen to, co agent v kódu nenajde sám.
4. Krok 3 po jednom: git switch -c c3-agents-md · git add AGENTS.md · git commit -m "Add AGENTS.md"

KOSTRA (ANGLICKY JAKO KÓD A README)

```
# AGENTS.md
## Build and test
- Everything: <one command>
- Fast loop: <target> and <ctest filter>
## Dependencies
- <where they come from, what not to add>
## Conventions
- <standard, naming, warnings>
## Domain rules (not visible in the code)
- <what the team knows>
## Do not touch without asking
- <files and why>
## Done means
- <commands that must pass>
```

Hotovo, když: AGENTS.md je v commitu ve vaší větvi a každý příkaz v něm jste sami spustili.

20 min

Po: stejná úloha, nová session

AGENTS.md se načte jen při startu. Nejdřív kontrolní otázka.

nová session; zkušební běh s AGENTS.md

```
› Jaké instrukce máš načtené? Vypiš nadpisy
  sekcí AGENTS.md a příkaz, kterým ověříš
  svou změnu. Nic neměň.

# pak stejná úloha; příkazy agenta v pořadí
rg --files -g 'AGENTS.md' -g '*qc*' …
git status --short && cat AGENTS.md && …
python scripts/check_all.py debug   # chyba
python scripts/check_all.py debug   # opraveno
python scripts/check_all.py tidy    # nález
git diff --check && git diff --stat …
python scripts/check_all.py debug
python scripts/check_all.py tidy    # čisto
```

PROČ TO TAM JE

1. **Kontrolní otázka**
   když agent vaše sekce nevyjmenuje, úlohu nepouštějte
2. **Nejdřív AGENTS.md**
   pravidla si přečetl dřív než kód
3. **Červená, pak zelená**
   debug podle „Done means“
4. **tidy z AGENTS.md**
   bez AGENTS.md ho agent nespustil

Zapište totéž co v kroku 1 a porovnejte. Rozdíl je výsledek cvičení.

Před a po: zkušební běh

Stejná úloha control_zone, čistý repozitář, jednou bez AGENTS.md a jednou s ním

	Bez AGENTS.md	S AGENTS.md (vzorové řešení)
SD = 0 a SD = −1	použil z_score: tiše vrací zónu 0	vyhodí std::invalid_argument
Hodnota přesně na mezi	120 → zóna 1, 130 → zóna 2	totéž, ostré meze
clang-tidy	nespustil; kontrola po běhu: tidy selže	spustil, opravil nález, znovu: projde
Spuštěné příkazy	4, ověření jen debug	8: debug, tidy, git diff --check
Doba, tokeny vstupu	45 s, 101 tis.	49 s, 204 tis.

AGENTS.md změnil to, co v kódu není vidět: pravidlo o SD a tidy v definici hotovo.

Reference: řádky, které dělají rozdíl

git diff c3-agents-md origin/reference/agents-md: výňatek z kořenového AGENTS.md

reference/agents-md:AGENTS.md (zkráceno)

```
## Build and test
- Everything, one command: `python scripts/check_all.py debug`
- Before you say a change is done, also run:
  `python scripts/check_all.py tidy`
- Any change that touches memory ownership, containers
  or lifetimes (rack, queue, scheduler, parser): also asan
- Fast loop: `--target labsched_tests`, `ctest -R <Suite>`
## Dependencies
- If the build fails on a dependency, stop and report
  which one; do not work around it.
## Conventions
- No new warnings. Do not add `-Wno-*` flags or
  `NOLINT` comments to make a build pass.
## Do not touch without asking
- `.clang-tidy` files: exceptions may only be removed.
```

PROČ TO TAM JE

1. **Definice hotovo jako příkaz**
   tidy před „hotovo“, ne až v pipeline
2. **Podmíněné pravidlo**
   asan jen při změně paměti: chytí zítřejší chybu
3. **Rychlá smyčka**
   sekundy místo minut
4. **Zastavit a nahlásit**
   jinak agent chybu závislosti „obejde“
5. **Zákaz zkratky**
   jinak agent varování umlčí
6. **Jednosměrné pravidlo**
   výjimky smí jen ubývat

Porovnejte se svým souborem: co máte navíc a co chybí. Rozdíl je materiál pro tým.

Skill: zabalený opakovaný postup

.agents/skills/<jméno>/SKILL.md

```
---
name: new-gtest-suite
description: Use when adding a new GoogleTest
  file to sample-service.
---
1. Put the file in tests/<library>/.
2. Add it to the suite in tests/CMakeLists.txt.
3. Build only <library>_tests, run ctest -R.
4. Check that ctest lists the new tests:
   ctest --preset debug -N -R <Suite>
```

- **description**
  spouštěč: kdy skill použít
- **Kde**
  .agents/skills v repu, ~/.agents/skills osobně
- **Načítání**
  model vidí jméno a popis, celý SKILL.md až při použití
- **Volání**
  $new-gtest-suite, nebo podle popisu
- **Jak vzniká**
  opakuji se, sepíšu, vyzkouším, sdílím přes MR

Postup, který jinak vysvětlujete každému novému kolegovi. A každé nové session.

Instrukce jako architektura

Co se načte vždy, co na vyžádání a kdo to vlastní

VŽDY V KONTEXTU

platíte za to v každé zprávě, držte to malé

AGENTS.md (všechny vrstvy)

build, testy, konvence, mapa

tým přes MR

popisy skillů

kdy který skill použít

autor skillu

popisy nástrojů MCP

co nástroj umí a kdy ho volat

správce napojení

NA VYŽÁDÁNÍ

načte se, jen když to úloha potřebuje

SKILL.md

celý postup krok za krokem

autor skillu

references/

podklady, na které skill odkáže

autor skillu

scripts/

spustí se, číst je nemusí

autor skillu

výsledky nástrojů MCP

data až při volání

správce napojení

Pravidla pro architekta

1. Detail patří do skillu nebo reference
2. Sdílená vrstva má vlastníka, mění se přes MR
3. Popis skillu je jeho spouštěč
4. Změnu testujte: stejná úloha před a po

AGENTS.md je rozcestník, skilly jsou příručky. Do kontextu jde jen to, co úloha potřebuje.

Skill add-service-command

Postup, jak do služby přidat nový příkaz

1 Napsat SKILL.md	2 Commit, nová session	3 Ověřit /skills

CO UDĚLAT

1. Vytvořte .agents/skills/add-service-command/SKILL.md: kde CommandProcessor rozděluje příkazy, kam patří logika, jak vypadá chyba použití, kam testy, README.
2. Soubor založte sami: .agents je pro agenta v sandboxu jen pro čtení.
3. Commit, pak nová session.
4. Ověřte, že ho Codex vidí: /skills, nebo zadání začínající $add-service-command.

.agents/skills/add-service-command/SKILL.md

```
---
name: add-service-command
description: <kdy se má skill použít;
  podle tohoto textu ho agent vybírá>
---
1. <krok>
2. <krok>
3. <jak ověřit, že je hotovo>
```

Hotovo, když: Codex v nové session skill vidí v /skills a použije ho.

15 min

BLOK 4 ZE 7 · ČÁST 1

Od požadavku po merge request, část 1

04

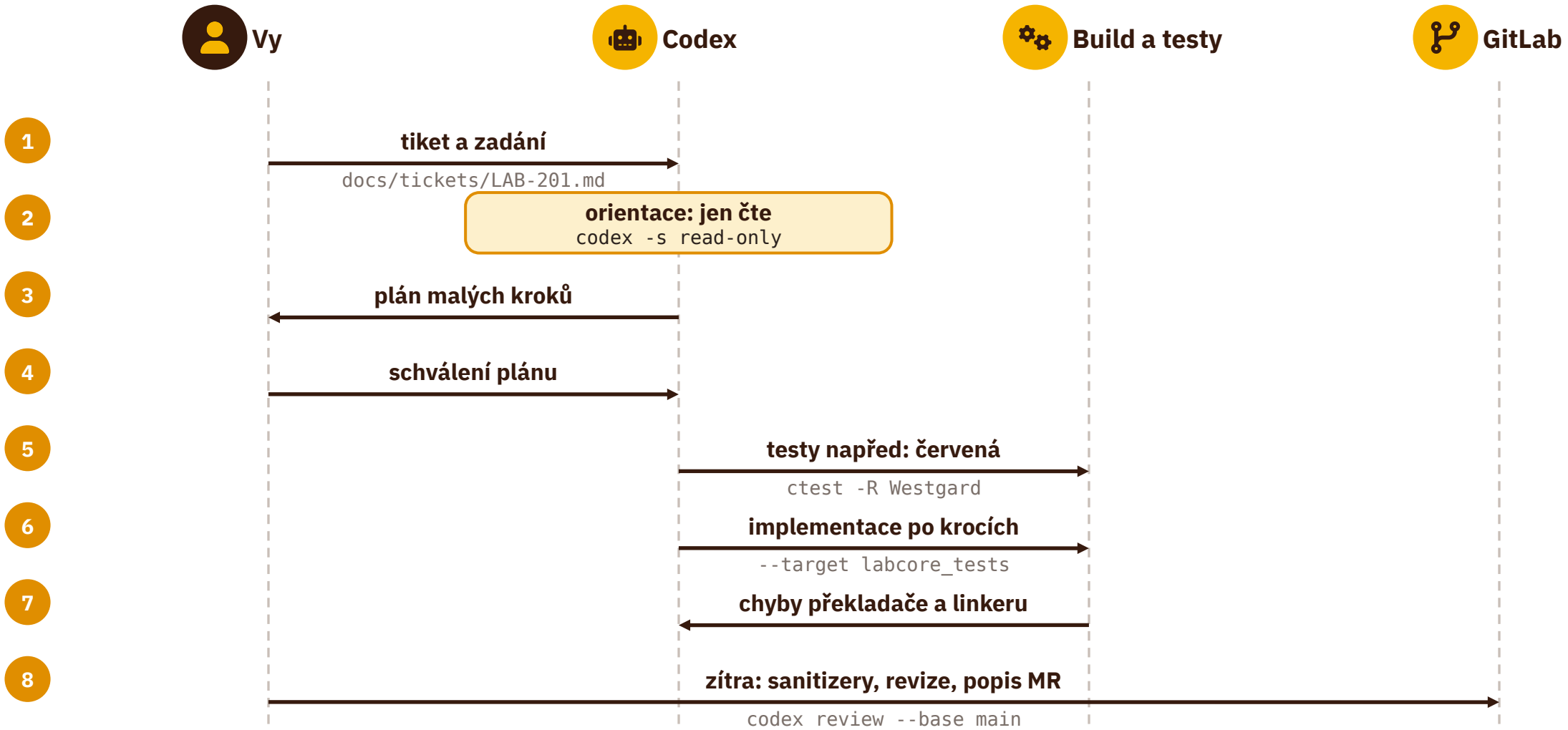
Orientace, plán, testy napřed, implementace malými kroky

150 MINUT · C4

Postup kurzu

Od tiketu po merge request

Kdo co dělá. Dnes kroky 1 až 7, zítra zbytek.



Orientace v cizím nebo starším kódu

První krok každé změny: agent čte, vy ověřujete

CO TO JE

Agent projde kód, vysvětlí architekturu a neznámou část, u každého tvrzení dá soubor a řádek. Nic nemění.

1 Mapa knihoven → 2 Vstupní body → 3 Neznámá část → **4 Podivnosti** → 5 Otázky

AGENT ZVLÁDNE

- Najít volající a závislosti rychleji než člověk
- Vysvětlit C styl, makra, staré formáty
- Vyjmenovat, co vypadá jako chyba

AGENT POKAZÍ, KDYŽ HO NEHLÍDÁTE

- „Opraví“ podivnost, která je záměr
- Domyslí si chování bez odkazu na řádek
- Začne refaktorovat, když má jen číst

DO ZADÁNÍ NEBO AGENTS.MD

```
Nic neměň.
U každého tvrzení
soubor a řádek.
Podivnosti jen
vyjmenuj, neopravuj.
Na konci tři místa,
kde si nejsi jistý.
```

Odkaz na řádek je kontrola: neexistující odkaz prozradí domyšlenou odpověď.

Kód s otázkami: legacy parser

src/labio/legacy/legacy_result_parser.cpp, výňatek se skutečnými čísly řádků

legacy_result_parser.cpp

```
18  static char g_fld[10][80];
19  static int g_nfld;
    ...
31        if (j < 79) {
    ...
62    if (line[1] == ';') {
63      d = ';';
64      fw = 1;
    ...
75        if (g_fld[0][0] == 'C') return LR_E_TYPE;
    ...
87      strncpy(r->assay, g_fld[3], 7);
    ...
106       else if (strcmp(p, "OK") == 0) { /* ignore */ }
    ...
117     r->pos = atoi(g_fld[1]);
119     r->errcode = atoi(g_fld[3]);
```

1 2 3 4 5 6

OTÁZKY DO MÍSTNOSTI

1. **Kolik vláken to snese?**
   statické buffery: parser není reentrantní
2. **Co se stane s polem o 100 znacích?**
   tiše se ořízne na 79
3. **Kolik formátů firmwaru tu je?**
   v1 se středníky, v2 se svislítky
4. **Smí v1 posílat kontroly?**
   C záznam ve v1 je chyba typu
5. **Co znamená příznak OK?**
   ignoruje se, neznámý příznak je chyba
6. **Co udělá atoi s „abc“?**
   vrátí 0 bez chyby: E|abc|… projde

Podivnost není automaticky chyba. Agent ji má najít a pojmenovat, rozhodujete vy.

Orientační zadání a odpověď, kterou lze ověřit

Agent jen čte. Každé tvrzení má číslo řádku.

ZADÁNÍ (codex -s read-only)

```
Vysvětli, co dělá lr_parse v
src/labio/legacy/legacy_result_parser.cpp
a jaké formáty řádků přijímá.

Nic neměň. U každého tvrzení uveď
soubor a řádek.

Vyjmenuj chování, které vypadá jako
chyba, ale může být záměr.
Nic neopravuj.
```

DOBRÁ ODPOVĚĎ (ZKRÁCENO, ŘÁDKY SEDÍ)

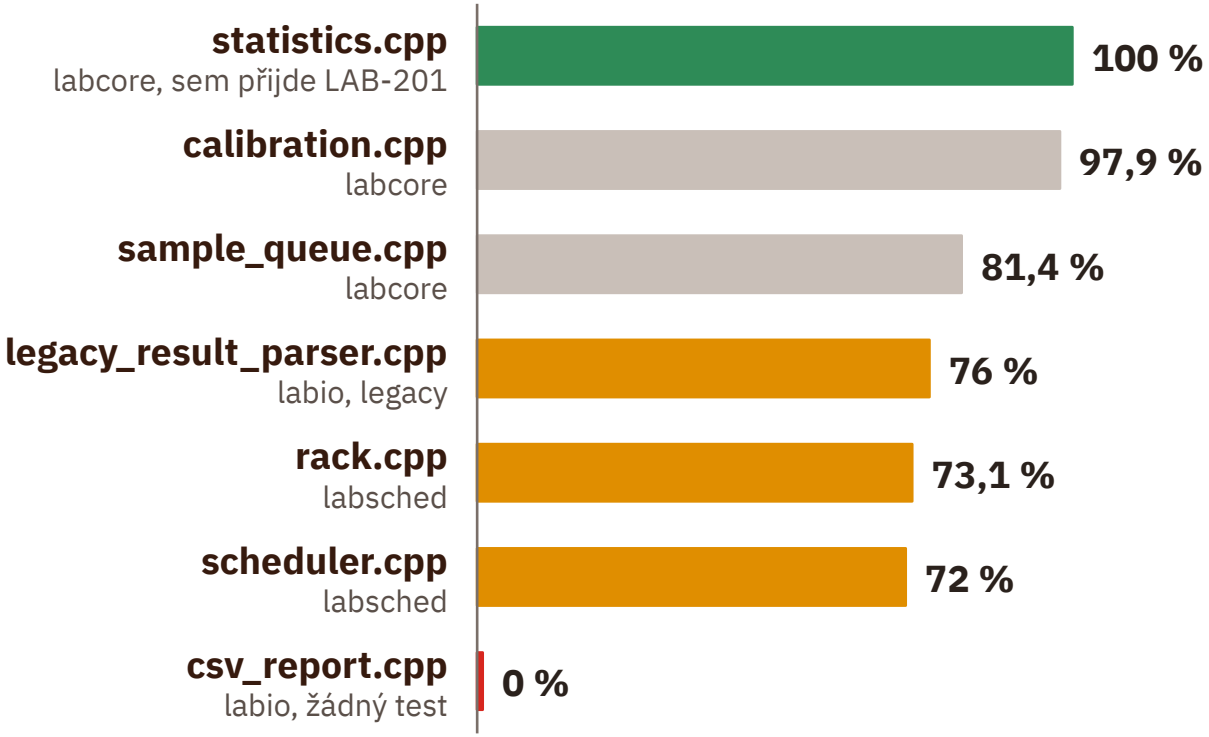
```
- v2: R|, C|, E| a H| (ř. 62 až 66)
- v1: R; bez kontrolních záznamů (ř. 75)
- pole nad 79 znaků se tiše ořízne
  (ř. 31)
- kód testu max. 7 znaků (ř. 87)
- příznak OK se ignoruje (ř. 106)
- E záznam: atoi bez kontroly
  (ř. 117 a 119)
- globální buffery, není reentrantní
  (ř. 18)
```

Odpověď kontrolujte namátkou: otevřete dva odkazy. Když nesedí, nevěřte ani zbytku.

Kde chybí testy

Řádkové pokrytí po souborech, preset coverage

ŘÁDKOVÉ POKRYTÍ TESTY (%)



Co z toho plyne

- labcore je dobře pokrytý: nová funkce má kam navázat
- Kde pokrytí chybí, agentovi nevěřte bez nových testů
- Pokrytí neříká, jestli testy testují správnou věc
- Proč má csv_report 0 % a co to stojí: zítra

Zdroj: Měřeno 26. 10. 2026: preset coverage a scripts/coverage_report.py (GCC 12.2, gcov), sample-service main

Před změnou zjistěte, jakou síť pod sebou máte. Agent to umí změřit za vás.

LAB-201: co tiket rozhodl a co ne

Analýza s agentem: nechte ho vyjmenovat díry, rozhodněte je vy

Tiket rozhodl

Mez
„za mezí“ = ostře větší než mez

Průměr
hodnota přesně na průměru přeruší řadu 10-x

Kdy hlásit
na každém indexu, kde okno pravidlo splní; 1-2s vždy

Pořadí
podle indexu, pak podle pořadí pravidel v tabulce

Tiket neříká

NaN a nekonečno ve vstupu
chyba, nebo přeskočit?

SD = NaN
podmínka SD <= 0 ji nezachytí

Velmi dlouhá řada
výkon a paměť nejsou zadané

Napojení na službu
mimo rozsah, samostatný tiket

Co tiket neříká, nesmí agent rozhodnout potichu. Buď to doplníte do tiketu, nebo agent vrátí otázku.

Smí evaluate_westgard použít z_score?

Existující funkce v labcore, skutečný kód a test

src/labcore: statistics.hpp, statistics.cpp, statistics_test.cpp

```
// statistics.hpp, řádky 29 a 30
// z-score of value against a target mean and standard deviation.
[[nodiscard]] double z_score(double value, double target_mean,
                             double target_sd) noexcept;

// statistics.cpp, řádky 68 až 73
double z_score(double value, double target_mean,
               double target_sd) noexcept {
  if (target_sd <= 0.0) {
    return 0.0;
  }
  return (value - target_mean) / target_sd;
}

// statistics_test.cpp, řádek 54
EXPECT_DOUBLE_EQ(z_score(7.0, 10.0, 0.0), 0.0);
```

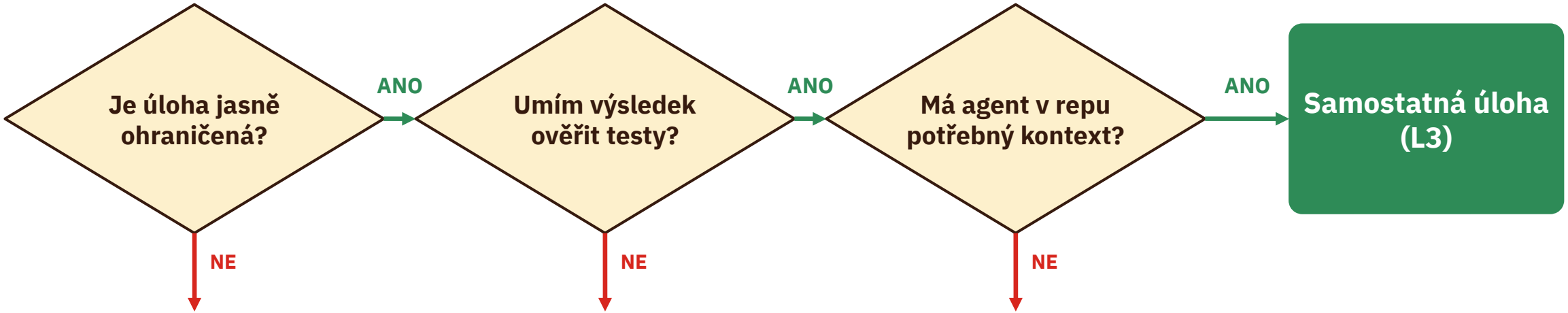
OTÁZKY DO MÍSTNOSTI

1. **Použít, nebo napsat znovu?**
   použít ano, ale SD zkontrolovat napřed
2. **Co vrátí pro SD = 0?**
   nulu, ne výjimku. Tiket chce výjimku.
3. **Smí funkce s noexcept vyhodit výjimku?**
   ne: evaluate_westgard musí SD ověřit sama
4. **Chyba, nebo záměr?**
   test chování zachytil: neměnit bez dohody

Agent, který z_score použije bez čtení, splní build i testy. Jen ne řádek „SD = 0“.

Dát úlohu agentovi?

Třicet sekund rozhodování ušetří hodinu



Tři ano = delegujte. Jedno ne = nejdřív to opravte.

Plán před implementací

Oprava na papíře je levná, v kódu drahá

ZADÁNÍ (STÁLE JEN ČTENÍ)

```
Přečti tiket LAB-201 a src/labcore.
Nic neměň. Navrhni plán malých kroků.
U každého kroku uveď:
- soubory, které změníš
- test, který krok dokáže
- příkaz, kterým to ověříš
Na konci vypiš otázky k nejasnostem
v tiketu.
```

PLÁN, KTERÝ CHCETE VIDĚT (ILUSTRATIVNÍ)

```
1. qc_rules.hpp: Violation, deklarace
   ověření: build labcore
2. qc_rules_test.cpp + tests/CMakeLists.txt
   případy z tiketu, ctest -R Westgard
   (červená)
3. 1-2s, 1-3s, kontrola SD
4. 2-2s, R-4s
5. 4-1s, 10-x
6. preset debug a tidy

Otázky: hodnota přesně na mezi?
10-x a hodnota rovná průměru?
```

Plán schvalujete vy. Teprve pak dostane agent právo zapisovat.

Testy napřed

Nejlepší obrana proti „skoro hotovo“

HODNOTY PÍŠE ČLOVĚK

tabulka z tiketu, dřív než kód

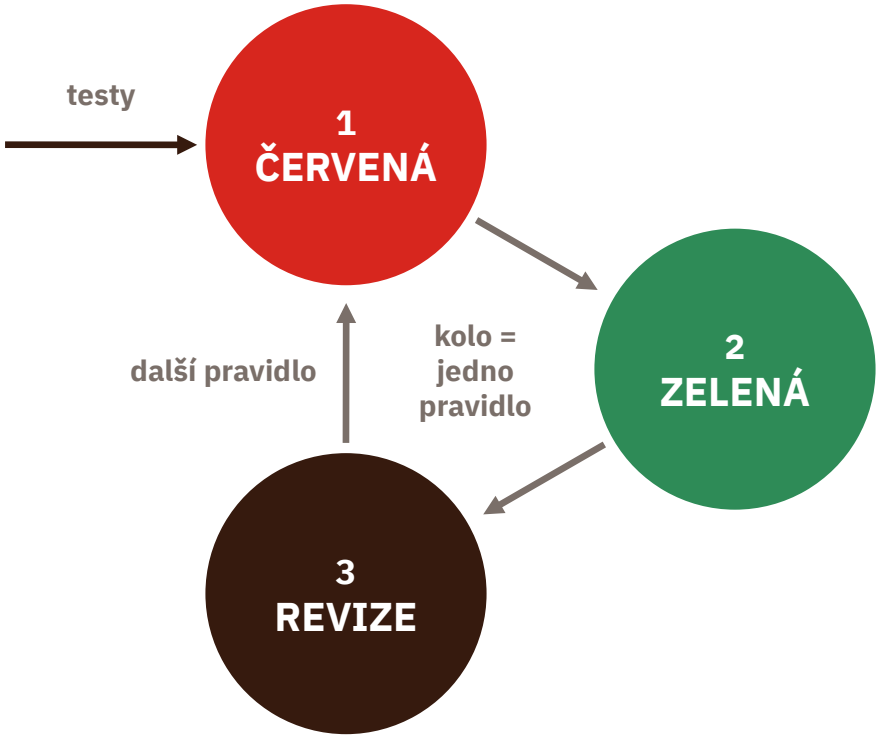
```
100, 125
→ 1-2s@1

120 (přesně +2 SD)
→ nic

131
→ 1-3s@0, 1-2s@0

SD = 0
→ invalid_argument
```

13 řádků = kritérium hotovo



1 ČERVENÁ

testy padají ze správného důvodu

```
ctest -R Westgard
2 z 5 padá: OneBeyond2Sd
čeká 1-2s@1, kostra vrací []
```

2 ZELENÁ

agent píše, dokud neprojdou

```
78 testů zelených
akceptace tiketu 14 z 14
```

3 REVIZE

člověk čte testy i kód

```
git diff cviceni/04-start -- tests/
žádné očekávání změněné
```

Pořadí je pojistka: hodnoty od člověka vzniknou dřív než kód, který by je mohl ovlivnit.

Agent navrhne scénáře, hodnoty potvrdíte vy

Generování testovacích scénářů a pokrytí kritických případů

ZADÁNÍ

```
Navrhni testovací scénáře pro
evaluate_westgard podle tiketu LAB-201.
U každého: co testuje a proč je kritický.
Hodnoty nevymýšlej, popiš jen vstup.
Zvlášť vypiš hraniční případy a u každého,
jestli ho tiket rozhoduje.
Nic neimplementuj.
```

NÁVRH SCÉNÁŘŮ (ILUSTRATIVNÍ)

```
- prázdný vstup
- jedna hodnota za +2 SD (varování)
- jedna hodnota za -3 SD
- dvě po sobě za +2 SD, totéž pod -2 SD
- +2 SD a hned -2 SD (R-4s)
- čtyři za +1 SD, čtyři za -1 SD
- deset nad průměrem, deset pod

# hraniční případy
- přesně na mezi: tiket rozhoduje (ostře)
- na průměru v řadě 10-x: rozhoduje
- SD = 0, SD záporná: rozhoduje (výjimka)
- SD = NaN: tiket neříká
```

Scénáře může vymyslet agent. Očekávané hodnoty do tabulky píše člověk.

Tabulka očekávaných výsledků: LAB-201

Průměr 100, SD 10. Každý řádek je jeden test.

HODNOTY	OČEKÁVANÉ PORUŠENÍ	PROČ TEN ŘÁDEK
`prázdný vstup`	nic	prázdný výsledek
`100, 125`	1-2s · 1	jen varování
`131`	1-3s · 0 · 1-2s · 0	1-2s se hlásí i vedle 1-3s
`121, 122`	2-2s · 1 · 1-2s · 0 · 1-2s · 1	dvě po sobě nad +2 SD
`121, 78`	R-4s · 1 · 1-2s · 0 · 1-2s · 1	rozpětí přes 4 SD
`111, 112, 113, 114`	4-1s · 3	čtyři po sobě nad +1 SD
`111 až 115`	4-1s · 3 · 4-1s · 4	hlásí se na každém indexu
`101 až 110`	10-x · 9	ne 4-1s: 107 až 109 pod +1 SD
`101 až 109, pak 100`	nic	průměr řadu přeruší
`120 (přesně +2 SD)`	nic	**na mezi není „za mezí“**
`130 (přesně +3 SD)`	1-2s · 0	**jen 1-2s, ne 1-3s**
`110, 110, 110, 110`	nic	**přesně +1 SD: ne 4-1s**
`SD = 0 nebo záporná`	std::invalid_argument	z_score by vrátil 0

Indexy od 0. Hnědě porušení, oranžově varování 1-2s, žlutě hrany. Spočítáno skriptem podle tiketu.



Hrana je řádek tabulky. Tabulka je kritérium hotovo: co v ní není, nikdo nezkontroluje.

Tabulka jako data v GoogleTestu

Parametrizovaný test a první skutečná chyba překladu

tests/labcore/qc_rules_test.cpp (výňatek)

```
struct Case {
  std::string name;
  std::vector<double> values;
  std::vector<std::string> expected;  // "1-2s@1"
};
class Westgard : public testing::TestWithParam<Case> {};

TEST_P(Westgard, MatchesExpectedTable) {
  EXPECT_EQ(flatten(evaluate_westgard(GetParam().values,
                                      100.0, 10.0)),
            GetParam().expected);
}
INSTANTIATE_TEST_SUITE_P(Table, Westgard, testing::Values(
  Case{"Empty", {}, {}},
  Case{"OneBeyond2Sd", {100, 125}, {"1-2s@1"}},
  Case{"ExactlyTwoSdIsNotBeyond", {120}, {}}),
  [](const auto& info) { return info.param.name; });
```

CO TU JE

1. **Očekávání jako data**
   „pravidlo@index“, čitelné i v hlášce o chybě
2. **Jeden test, mnoho případů**
   každý řádek tabulky má vlastní jméno
3. **Hrana z tabulky**
   přesně +2 SD: nic se nehlásí
4. **První překlad: chyba**
   -Werror=shadow: 'info' zastiňuje parametr makra INSTANTIATE_TEST_SUITE_P. Oprava v kódu: p, ne -Wno-shadow.

Chyba překladu není selhání. Je to první zpětná vazba, kterou agent dostane zadarmo.

Správná červená, falešná zelená

Skutečné výstupy: testy proti prázdné implementaci a test mimo CMakeLists

PRÁZDNÁ IMPLEMENTACE: ČERVENÁ

```
$ ctest --preset debug -R Westgard
1/5 WestgardInput.ZeroSdThrows .. Passed
2/5 Table/.../Empty ............. Passed
3/5 Table/.../OneBeyond2Sd .... ***Failed
4/5 Table/.../TenOnOneSide .... ***Failed
5/5 Table/.../ExactlyTwoSd... ... Passed

60% tests passed, 2 tests failed
out of 5
```

TEST MIMO tests/CMakeLists.txt: ZELENÁ

```
# qc_rules_test.cpp leží v tests/labcore/,
# v tests/CMakeLists.txt chybí

$ ctest --preset debug
100% tests passed, 0 tests failed
out of 49

$ ctest --preset debug -R Westgard
No tests were found!!!
```

Test, který projde i na prázdném kódu, nic nedokazuje. Zelený ctest bez nových testů také ne.

LAB-201 po krocích

Každý krok: jedno pravidlo, build, testy, commit

LAB-201
tiket a tabulka očekávaných výsledků

Plán z kroku 1
plán: hlavička, testy, pak pravidla po skupinách

1-2s, 1-3s
qc_rules.cpp, kontrola SD
- build
- testy
- diff

hotovo, commit

2-2s, R-4s
qc_rules.cpp
- build
- testy
- diff

hotovo, commit

4-1s
qc_rules.cpp
- build
- testy
- diff

běží

10-x
qc_rules.cpp
- build
- testy
- diff

čeká

Záchranná síť: tabulka testů z tiketu, po každém kroku ctest -R Westgard

Krok jedno pravidlo → **Build** --target labcore_tests → **Testy** ctest -R Westgard → **Commit** malý diff, čitelná zpráva

Když se krok rozbije, vracíte se o jeden commit, ne o celé odpoledne.

Diff, který nesmí projít

Agent „opravil“ test, aby byl zelený

git diff (ilustrativní)

```
@@ src/labcore/src/qc_rules.cpp @@
   const double z = (values[i] - target_mean) / target_sd;
-  if (std::abs(z) > 2.0) {
+  if (std::abs(z) >= 2.0) {                                  1
     out.push_back({"1-2s", i});

@@ tests/labcore/qc_rules_test.cpp @@                        3
     Case{"TenOnOneSide", {101, 102, 103, 104, 105,
                           106, 107, 108, 109, 110}, {"10-x@9"}},
-    Case{"ExactlyTwoSdIsNotBeyond", {120}, {}}),
+    Case{"ExactlyTwoSdIsNotBeyond", {120}, {"1-2s@0"}}),     2
```

NA CO SE DÍVAT

1. **Hrana rozhodnutá potichu**
   > se změnilo na ≥ bez otázky
2. **Hodnota z tabulky změněná**
   jméno testu teď lže samo sobě
3. **Diff testů čtěte první**
   změna očekávání = stop, ne merge

Agent nesmí měnit očekávané hodnoty. Když test podle něj nesedí, má se zeptat.

Chyba linkeru jako vstup pro agenta

Hlavička, implementace i testy existují. Přesto linker selže.

```
$ cmake --build --preset debug
FAILED: tests/labcore_tests
qc_rules_test.cpp:13: undefined reference to
  `lab::qc::evaluate_westgard(
     std::span<double const, 18446744073709551615ul>,
     double, double)'
collect2: error: ld returned 1 exit status

# src/labcore/CMakeLists.txt:
# add_library(labcore ... src/sample_queue.cpp)
#   qc_rules.cpp v seznamu chybí
```

- **Překlad prošel**
  hlavička i testy jsou v pořádku
- **Linker ne**
  qc_rules.cpp není v add_library(labcore ...)
- **Oprava**
  jeden řádek v src/labcore/CMakeLists.txt
- **Pro agenta**
  celý výstup, ne „nejde to přeložit“

Undefined reference téměř nikdy není chyba v kódu. Je to chyba v CMake.

Pět hlášek a co znamenají

Všechny jste dnes viděli na sample-service

Hláška	Typická příčina u agenta	Kde opravit
fatal error: ...: No such file or directory	špatná hlavička, include cesta, <format> v GCC 12	název souboru, target_include_directories
error: 'x' is not a member of 'y'	neexistující API nebo novější standard	skutečné jméno, standard v zadání
undefined reference to ...	soubor mimo add_library, chybí target_link_libraries	CMakeLists, ne kód
error: ... [-Werror=shadow]	varování, které projekt bere jako chybu	kód, ne -Wno-* ani NOLINT
No tests were found!!!	test mimo tests/CMakeLists.txt, špatný filtr	registrace testu, ctest -N

Chyba překladače je pro agenta nejlepší vstup. Dejte mu celý výstup a nechte ho, ať to opraví sám.

Jak agentovi předat chybu

Agent opravuje jen tak dobře, jak dobrý vstup dostane

1	**Ať si build spustí sám**	Výstup má z první ruky, ne přepsaný.	*cmake --build --preset debug*
2	**První chyba první**	Další bývají jen následky.	*ne 500 řádků logu*
3	**Celá hláška**	Soubor, řádek, typ a poznámky překladače.	*ne „nejde to“*
4	**Oprava ve správné vrstvě**	CMakeLists, include, nebo kód.	*ne -Wno-* a NOLINT*
5	**Točí se v kruhu**	Stejná chyba podruhé znamená stop.	*zpět k plánu*

Když agent stejnou chybu dvakrát neopraví, problém je v zadání nebo v plánu, ne v kódu.

CVIČENÍ C4

80 MINUT

LAB-201 od tiketu po zelené testy

Pět kroků v jedné session, po každém se díváte, co agent udělal

CÍL

Projít s Codexem postup na jedné funkci: orientace, plán, testy napřed, implementace malými kroky, statická analýza a vlastní revize diffu.

`cviceni/04-tdd-westgard, větev cviceni/04-start`

POSTUP

1. Orientace a plán (10 min)
2. Testy napřed podle tabulky (15 min)
3. Implementace malými kroky (20 min)
4. Kvalita: debug a tidy (10 min)
5. Vlastní revize diffu (5 min)

ODEVZDÁTE

Větev lab-201: testy podle tabulky, implementace, zelený debug i tidy.

HOTOVO, KDYŽ

každý řádek tabulky má test, všechny testy projdou, debug i tidy projdou a diff je jen v souborech z tiketu.

ictPRO

Orientace a plán

Zatím nic neměnit: místa změny, plán malých kroků, nejasnosti

CO UDĚLAT

1. Čistý stav po jednom: git restore . · git clean -fd
2. git switch cviceni/04-start
3. git switch -c lab-201
4. python scripts/check_all.py debug: 49 testů zelených
5. codex --sandbox workspace-write; všechny kroky v jedné session.
6. Projděte plán. Nesouhlas řekněte agentovi teď, ne až v diffu.

ZADÁNÍ PRO CODEX

```
Přečti docs/tickets/LAB-201.md. Zatím nic
neměň. Najdi v kódu místa, kterých se
změna týká (knihovna, CMake, testy,
konvence), a navrhni plán malých kroků;
u každého kroku uveď test, který ho
dokáže. Na konci vypiš, co v tiketu není
jednoznačné.
```

Hotovo, když: máte plán, se kterým souhlasíte, a seznam nejasností v tiketu.

10 min

ictPRO

Testy napřed podle tabulky

Hlavička, prázdná implementace a testy, které padají

CO UDĚLAT

1. Pod zadání vložte tabulku očekávaných výsledků ze ZADANI.md.
2. Padat mají asserty, ne překlad.
3. Sami zkontrolujte, že každý řádek tabulky má test s přesným očekáváním.
4. ctest --preset debug -N: jsou nové testy registrované?

ZADÁNÍ PRO CODEX

```
Napiš testy v GoogleTest do
tests/labcore/qc_rules_test.cpp podle tiketu.
Přidej jen tolik kódu, aby se testy
přeložily (hlavička a prázdná implementace),
a zapiš soubory do CMake. Testy musí pokrýt
nejméně případy z tabulky níže (průměr 100,
SD 10). Spusť testy cíleně (jen
labcore_tests) a ukaž, že padají ze
správného důvodu, ne kvůli chybě překladu.

<tabulka očekávaných výsledků ze ZADANI.md>
```

Hotovo, když: každý řádek tabulky má test, testy jsou registrované a padají na assertech.

15 min

Kontrolní bod: testy proti tabulce

Skutečný diff testů ze zkušebního běhu C4. Sedí každé očekávání?

git diff cviceni/04-start -- tests/labcore (výňatek)

```
@@ tests/labcore/qc_rules_test.cpp @@
+  WestgardCase{"AtUpperTwoSigma", {120}, {}},                    1
+  WestgardCase{"AtUpperThreeSigma", {130}, {{"1-2s", 0}}},
+  WestgardCase{"AtUpperOneSigma", {110, 110, 110, 110}, {}},
+  WestgardCase{"FiveBelow", {89, 88, 87, 86, 85},                2
+               {{"4-1s", 3}, {"4-1s", 4}}},
+  WestgardCase{"OverlappingPairs", {121, 122, 123},              3
+               {{"1-2s", 0}, {"1-2s", 1}, {"2-2s", 1},
+                {"1-2s", 2}, {"2-2s", 2}}},
```

OTÁZKY DO MÍSTNOSTI

1. **Sedí hrany s tabulkou?**
   120 nic, 130 jen 1-2s, 4× 110 nic: ano
2. **Případy navíc**
   pod průměrem: v tabulce nejsou, dobrý přídavek
3. **Kdo spočítal očekávání?**
   agent sám: přepočítejte ručně

Případy navíc jsou dobře. Jejich očekávání ale musí přepočítat člověk.

ictPRO

Implementace malými kroky

Pravidlo po pravidle, po každém kroku cílené testy

CO UDĚLAT

1. Sledujte, že agent po každém kroku spouští jen labcore_tests.
2. Chyby překladače a linkeru nechte agenta číst a opravit.
3. Když sáhne na test nebo mimo rozsah: zastavit.
4. Když hlásí rozpor testu s tiketem, rozhodujete vy.

ZADÁNÍ PRO CODEX

```
Implementuj evaluate_westgard po malých
krocích (pravidlo po pravidle) a po každém
kroku spusť cílené testy. Chyby překladače
čti a oprav sám. Testy neměň, pokud nejsou
v rozporu s tiketem; když jsou, zastav se
a řekni mi to.
```

Hotovo, když: všechny testy z tabulky procházejí a agent nezměnil žádné očekávání.

20 min

ictPRO

Kvalita: debug a tidy

Celý build a statická analýza, opravy bez zkratek

CO UDĚLAT

1. Celý build debug a tidy, ne jen cílené testy.
2. Nálezy tidy se opravují v kódu, ne přes NOLINT ani -Wno-*.
3. Sledujte, co agent mění kvůli varováním. Je to v rozsahu?
4. Tidy z čistého stavu trvá desítky sekund (blok 3).

ZADÁNÍ PRO CODEX

```
Spusť python scripts/check_all.py debug
tidy a oprav nálezy bez NOLINT
a bez -Wno-*.
```

Hotovo, když: python scripts/check_all.py debug tidy projde.

10 min

Vlastní revize: mění diff jen to, co tiket chce?

Minulý zkušební běh C4 (26. 10.): git diff --stat a git diff

git diff cviceni/04-start (výňatek)

```
$ git diff --stat cviceni/04-start
 src/labcore/CMakeLists.txt                |  1 +
 src/labcore/include/labcore/qc_rules.hpp  | 19 +++
 src/labcore/src/qc_rules.cpp              | 59 +++++
 tests/CMakeLists.txt                      |  3 +-
 tests/labcore/qc_rules_test.cpp           | 76 +++++

$ git diff cviceni/04-start -- tests/CMakeLists.txt
@@ sample_add_test_suite(labcore_tests @@
+  labcore/qc_rules_test.cpp
@@ sample_add_test_suite(labsched_tests @@
-target_link_libraries(labsched_tests PRIVATE lab::sched
-                      sample_service_app)
+target_link_libraries(labsched_tests PRIVATE
+                      sample_service_app)
```

PROČ TO TAM JE

1. **tests/CMakeLists.txt: 3 řádky**
   registrace testu je jeden. Co jsou ty další?
2. **Registrace testu**
   přesně to, co tiket chce
3. **Mimo rozsah**
   agent odebral lab::sched kvůli varování linkeru
4. **Testy dál prošly**
   přesto vrátit, nebo vědomě schválit

Testy i akceptace prošly. Přesto byl v diffu řádek, který do této změny nepatřil.

Kontrola před odevzdáním

Kritérium hotovo ze zadání C4 a kontrolní seznam revize

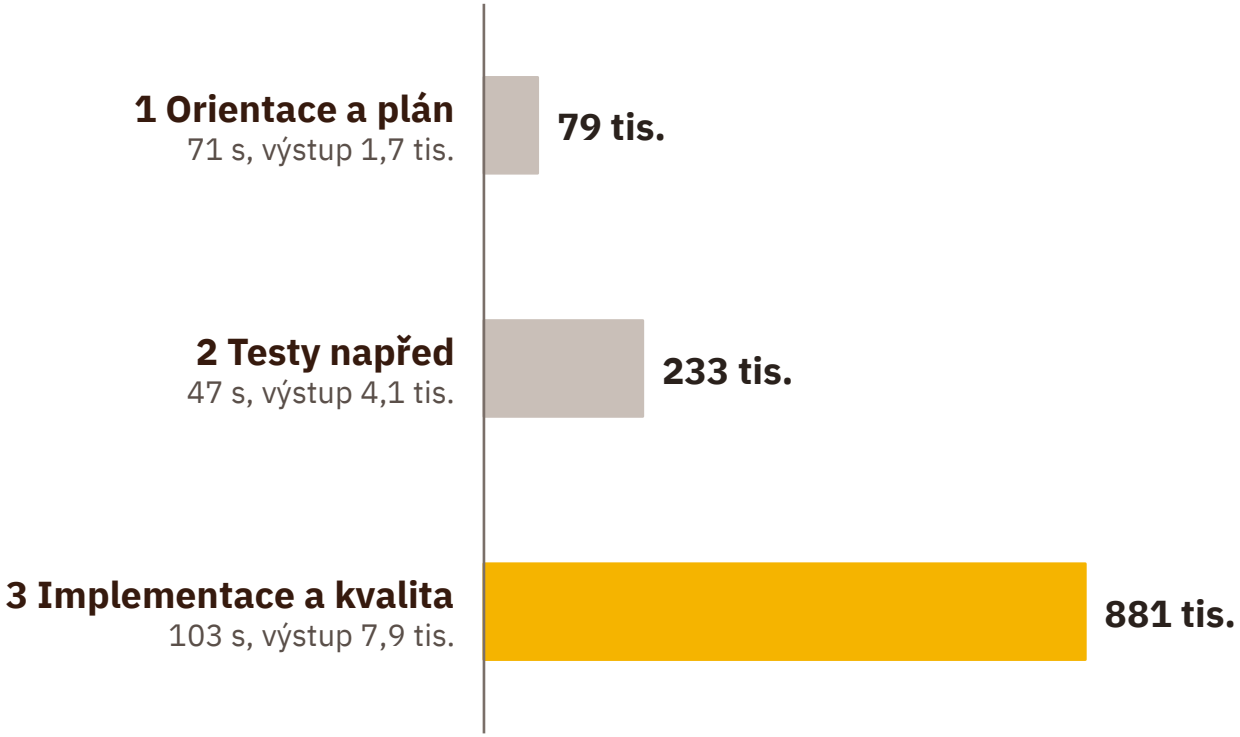
Kontrola	Jak ověřit
Každý řádek tabulky má test, žádné očekávání změněné	git diff cviceni/04-start -- tests/
Nové testy jsou registrované a procházejí	ctest --preset debug -N, pak ctest --preset debug
debug a tidy projdou	python scripts/check_all.py debug tidy
Diff jen v souborech z tiketu	git diff --stat cviceni/04-start
Použité API existuje, nic oslabeného (NOLINT, -Wno-*)	git diff cviceni/04-start

To je váš podpis pod změnou. Zítra k němu přidáme sanitizer, revizi v nové session a popis MR.

Zkušební běh C4: kde se platí

Tři zadání v jedné session; kroky 3 a 4 tu byly jedním zadáním

TISÍCE TOKENŮ NA VSTUPU PO KROCÍCH (MĚŘENO)



Co agent udělal

- 7krát cílený build labcore_tests, z toho 6krát s filtrem testů
- tidy jednou selhal, agent nález opravil a pustil znovu
- Výsledek: 78 testů zelených, akceptace tiketu 14 z 14
- V kroku 3 šlo 97 % vstupu z mezipaměti; výstup jen pár tisíc tokenů

Zdroj: Zkušební běh lektora 26. 10. 2026, codex exec v Codex CLI 0.163.0, výchozí model

Postup z dneška funguje i bez lektora. Náklady tvoří čtení dlouhé session, ne psaní kódu.

C4: co jste viděli

1. Kde jste agenta museli zastavit?
2. Změnil agent někdy test místo kódu?
3. Který krok vám ušetřil nejvíc času?



Tři věci z dneška

ictPRO

1

Zadání s hranami

Cíl, rozsah, omezení, kritérium hotovo. Hrany požadavku rozhodujete vy, ne agent.

2

Kontext v repozitáři

Krátký AGENTS.md ve vrstvách, presety a cílené testy.

3

Testy napřed

Tabulku píše člověk, agent z ní testy, pak kód.

Kterou z nich zkusíte zítra ráno ve vlastním repozitáři?

BONUS

Pro zájemce

Modely, srovnání nástrojů a kombinace



ZÁLOHA PODLE OTÁZEK

Modely v Codex CLI 0.163.0

Katalog modelů, který dodává samotné CLI (popisy přeložené)

Model	Popis v katalogu	Výchozí úsilí	Kdy sáhnout
gpt-6.1-sol	nejnovější model na programování a běžnou práci	low	běžná práce, výchozí model
gpt-6-astra	špičková inteligence pro nejnáročnější práci	low	nejtěžší úlohy, výrazně dražší
gpt-6-sol	předchozí generace modelu na běžnou práci	medium	předchozí generace
gpt-6-luna	rychlý a levný model na snazší úlohy	medium	rutina, hledání, souhrny

Model si vyberete přepínačem -m nebo v /model. Na dnešní úlohy stačí výchozí.

Pět nástrojů v jedné tabulce

Kde běží, co čtou a kde mají slabinu. Ověřeno na stránkách výrobců 26. 10. 2026.

Nástroj	Kde běží	Instrukce projektu	Hodí se na	Slabá stránka
Codex	CLI, VS Code, aplikace, cloud	AGENTS.md, skilly	samostatné úlohy s buildem a testy	cloud s GitLabem zatím jen v betě
GitHub Copilot	VS Code, JetBrains, CLI, GitHub	copilot-instructions.md; AGENTS.md ne všude	doplňování a agent v editoru	cloudový agent jen pro repozitáře na GitHubu
Cursor	vlastní editor, CLI, cloud, JetBrains	.cursor/rules, AGENTS.md	rychlé iterace v editoru	rozšíření editoru jen z Open VSX
Claude Code	CLI, VS Code, JetBrains, aplikace, web	CLAUDE.md, skilly; AGENTS.md jen bez CLAUDE.md	samostatné úlohy, více agentů	jen modely Claude
Devin Desktop (dříve Windsurf)	vlastní editor, cloudový Devin	.devin/rules, AGENTS.md	agent v editoru	v roce 2026 přejmenován, původní agent zrušen

Nástroj si nevybírejte podle modelu. Vybírejte podle toho, kde pracujete a co umíte ověřit.

Co přežije změnu nástroje

Stejné vrstvy, jiná jména. Žluté vrstvy jsou vaše investice.

	Codex	Claude Code	GitHub Copilot
Model mění se často	GPT	Claude	výběr z více výrobců
Kde běží smyčka a nástroje	CLI, aplikace, VS Code, JetBrains, web	CLI, aplikace, VS Code, JetBrains, web	VS Code, JetBrains, CLI, GitHub
Instrukce projektu pravidla týmu	`AGENTS.md`	`CLAUDE.md`	`copilot-instructions.md`, `AGENTS.md`
Opakované postupy zabalené know-how	skilly	skilly	skilly, soubory s prompty
Napojení GitLab, Jira, dokumentace	MCP	MCP	MCP
Oprávnění co smí bez ptaní	sandbox a schvalování	režimy oprávnění	schvalování nástrojů
Ověření build, testy, pipeline, revize	v repu	v repu	v repu

S instrukcemi, skilly a testy v repu je přechod na jiný nástroj levnější, než se zdá.

Kombinace podle typu práce

Doporučení pro C++ tým, ne dogma

1	**Nová funkce s testy**	Codex v CLI nebo ve VS Code, zadání s kritériem hotovo, testy napřed.	*LAB-201 dnes odpoledne*
2	**Starší kód bez testů**	Agent nejdřív jen čte a vysvětluje. Pak charakterizační testy, až potom změny.	*legacy parser, zítra*
3	**Rychlá otázka nad API**	Chat stačí. Firemní kód jen do nástroje, který firma povolila.	*bez kontextu repa*
4	**Revize merge requestu**	Druhý agent v nové session, bez kontextu autora.	*codex review, zítra*

Čím víc samostatnosti, tím víc musí být ověření v repozitáři: build, testy, sanitizery.

Otázky a diskuze

Zítra v 9:00 · jan@lakmoos.com

ictPRO

github.com/poli-cz/
sample-service-codex-kurz